



USENIX

THE ADVANCED COMPUTING
SYSTEMS ASSOCIATION

Do You Need a Receipt? Anonymous Credential Revocation at Continental Scale via Private Record Certification

Kasra EdalatNejad, Sebastian Faust, Jonas Hofmann, Philipp-Florens Lehwalder,
and Thomas Schneider, *Technical University of Darmstadt*

<https://www.usenix.org/conference/usenixsecurity26/presentation/edalatnejad>

**This paper is included in the Proceedings of the
35th USENIX Security Symposium.**

August 12–14, 2026 • Baltimore, MD, USA

ISBN 978-1-939133-58-8

**Open access to the Proceedings of the
35th USENIX Security Symposium
is sponsored by**



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

Do You Need a Receipt? Anonymous Credential Revocation at Continental Scale via Private Record Certification

Kasra EdalatNejad, Sebastian Faust, Jonas Hofmann, Philipp-Florens Lehwalder, Thomas Schneider

Technical University of Darmstadt, Germany

{edalat, schneider}@encrypto.cs.tu-darmstadt.de,

{sebastian.faust, jonas.hofmann1, philipp-florens.lehwalder}@tu-darmstadt.de

Abstract

A key challenge in digital credential systems is revocation, that is, the ability to invalidate credentials post-issuance and verify their status upon presentation. While anonymous credentials enhance privacy over classical credentials (e.g., by providing unlinkability), they complicate revocation. Existing revocation schemes for anonymous credentials often suffer from high client or verifier computation, long delays before revocation takes effect (e.g., epoch-based settings), or require updates to all users with each revocation.

We present an efficient, real-time revocation system for anonymous credentials with decentralized revocation authorities based on a novel primitive called *Private Record Certification* (PRC). PRC enables users to obtain a certificate for a record stored in a server-managed database without the servers learning which record was requested. This primitive is of independent interest, and we construct it by combining techniques from private information retrieval and secure multi-party computation. Our revocation scheme outsources its computation to the revocation authorities and has minimal overhead for clients and verifiers, while ensuring the communication costs are sublinear in the number of credentials. We build a prototype and demonstrate that our system achieves sub-second real-time latency for PRC requests, at a scale of over *1 billion* credentials, with an online operational cost of 2.5\$ per server for processing *1 million* PRC requests.

1 Introduction

As governments increasingly move public services online, electronic identification (eID) is becoming a critical national infrastructure, serving as the entry point for essential services like taxes, healthcare, mobility, and education. While eID systems ease administrative burdens for citizens, they also raise concerns about mass surveillance and the erosion of fundamental rights, such as privacy. European regulations [39] call for privacy-preserving digital identities that provide strong authenticity without enabling cross-context linkability. The

European Digital Identity (EUDI) Wallet initiative seeks to realize these legal requirements, but has major shortcomings and does not yet provide the demanded privacy guarantees [6]. Anonymous credentials [20, 22, 23] have long been known to achieve this securely and efficiently and future EUDI iterations may adopt anonymous credentials [38].

A central challenge in credential systems is *revocation*: the ability to invalidate credentials after issuance, and to verify a credential’s status during presentation. Privacy-preserving revocation is not a technical nicety; it is a two-sided safeguard. It protects citizens against identity theft, lost credentials, misuse, and surveillance, while empowering authorities to enforce accountability in a rights-respecting manner. For instance, suspended driver’s licenses or fraudulent professional certifications must be revoked without enabling pervasive profiling of citizens. Privacy can be important for revocation authorities too, especially when they are law enforcement or government agencies (e.g., for driver’s licenses). Solutions that rely on public revocation lists disclose the status of all credentials, exposing unnecessary data and creating legal risks. Demonstrating that strong privacy and real-time accountability can be achieved simultaneously is essential to counter the false trade-off between security and civil liberties, which often stalls adoption of privacy-preserving solutions.

While anonymous credentials offer strong privacy, they complicate revocation. Existing revocation schemes [5, 14, 16, 18, 21, 68, 74, 81] incur expensive client updates per revocation, high verifier costs, or delayed revocation via time-based epochs, which create “opportunity windows” for abuse of revoked credentials.

We study the requirements of a practical revocation scheme. Viable systems should: 1) scale for nation-wide deployment, supporting hundreds of millions of credentials, 2) support real-time revocation that eliminates the abuse window, and 3) achieve low latency with 4) minimal computational/communication overhead for clients and verifiers, enabling deployment on mobile devices and constrained networks. We align revocation privacy requirements with those of anonymous credential presentations. Since revocation in-

roduces authorities absent from classic credentials, we also aim to preserve the privacy of the credential status database, which we call *database privacy*. In particular, clients must prove ownership of a credential to learn its revocation status.

We construct a revocation scheme for anonymous credentials that achieves scalability, efficiency, and real-time revocation while maintaining strong privacy guarantees. Clients obtain a non-revocation token from a Revocation Authority (RA) and present it, along with their credential, to verifiers. We target an on-demand scenario where clients request a new token immediately before each credential presentation. Therefore, we must guarantee that non-revocation queries remain unlinkable. We require similar privacy guarantees for RAs as we do for issuers and verifiers in credential systems. This is needed, as otherwise the RA could track or discriminate against users based on their usage patterns, leaked attributes, or revocation status. To achieve this guarantee, we assume multiple RAs that share a replicated database in an anytrust setting, where unlinkability is preserved as long as at least one RA remains honest. The client does not need to know which one is honest. This is a realistic model, as in practice, the distributed RA can be realized as a consortium of different servers, e.g., operated by different governmental and non-governmental institutions.

As a first step toward a revocation scheme, we introduce a new primitive called Private Record Certification (PRC) that enables clients to privately obtain a certificate for a record in a server-managed database. Concretely, it provides a signature on a record $\mathbf{D}[\text{idx}]$ together with its index idx , while hiding the identity of the client, the index itself, and the record value. Clients can use this signature in standard zero-knowledge proofs to demonstrate their record satisfies a desired property. Beyond revocation, PRC is of independent interest for applications such as privacy-preserving proofs of credit score or account balance. Our PRC construction combines techniques from Private Information Retrieval (PIR) [25, 33] with Secure Multi-Party Computation (MPC) [44]. Specifically, we introduce a novel conversion from Boolean secret sharing to One-Hot Encoding (OHE) achieving theoretically minimal client-side complexity, logarithmic in the number of database records (i.e., the number of bits needed to represent the index in cleartext). We then combine our secret sharing operations in a new way with homomorphic commitments, enabling a seamless transition to asymmetric operations needed for building a non-revocation token via a (threshold) signature.

Contributions. We summarize our contributions below.

- ✓ We introduce Private Record Certification (PRC), a cryptographic primitive that enables clients to obtain a private certificate for a record in a server-managed database, along with formal definitions and security properties.
- ✓ We design a PRC scheme with minimal client and verifier cost complexity, and formally prove its security.
- ✓ We build a privacy-preserving revocation system for anonymous credentials, instantiated with our PRC.
- ✓ We implement and evaluate our PRC construction,

demonstrating that our revocation system achieves real-time revocation while scaling to *billions* of credentials with *sub-second latency* for revocation status lookups. Our solution is also cost-efficient: the online operation costs per PRC server for *one million queries* are only 2.5\$, based on AWS pricing.

1.1 Technical Overview

We provide a high-level technical overview of our solution next. We build a revocation scheme where a decentralized RA keeps a private status list $\mathbf{D}$, such that $\mathbf{D}[\text{idx}] \in \{0, 1\}$ is the validity status of the credential with a private index idx (see §6). When clients want to present one of their credentials, they need to prove that $\mathbf{D}[\text{idx}] = 1$, while 1) binding idx to the index embedded in their credential and 2) hiding idx to ensure unlinkability. To achieve this goal, we define a primitive called PRC that allows the client to interact with RAs to privately obtain a signed commitment linking $(\text{idx}, \mathbf{D}[\text{idx}])$, also called a non-revocation token. When clients present their credential, they present a non-revocation token and use zero-knowledge proofs to link the index in the credential to the token (i.e., proof of discrete log equality) and reveal the record value $\mathbf{D}[\text{idx}] = 1$. The verifier validates a) the credential presentation, b) the RA's signature on the token, and c) the ZK-proof proving the token corresponds to the credential. Since PRC provides *unforgeability*, the verifier can be certain that the revealed revocation status belongs to presented credential.

PRC. We define and construct a PRC scheme in §5, where given a private index, the PRC servers compute and then sign a commitment to $(\mathbf{D}[\text{idx}], \text{idx})$ that binds an index to its record value. We design a PRC protocol in the following steps: 1) the client Boolean secret shares its index among PRC servers. 2) the servers convert their boolean shares of idx to arithmetic shares and compute arithmetic shares of the database record $\mathbf{D}[\text{idx}]$. This step has a similar nature as PIR, and we design a custom optimized variant of it, which we discuss below. 3) Each server commits to its arithmetic share of the index and record value via a homomorphic commitment like Pedersen. 4) The PRC servers aggregate the homomorphic commitment. As we deliberately choose the same group order between our commitment and arithmetic sharing, this aggregation acts as a secret reconstruction without revealing the cleartext values to PRC servers. 5) The PRC servers sign the final commitment.

We extend this scheme in §5.4 to achieve database privacy, by requiring clients to add a ZK-proof (similar to the proof used for presenting a non-revocation token) showing that they have an authorization for the requested index.

Record retrieval. In step 2) of our PRC scheme, the PRC servers - each holding a replicated database $\mathbf{D}$ - obtain an arithmetic sharing of the record $\mathbf{D}[\text{idx}]$, where idx is the client's private index. This step resembles multi-server PIR schemes [25] which (Boolean) secret share the OHE of the client's index, an all-zero vector with a single 1 at position idx . The servers

then compute the inner product of this vector and the database and return the result to the client to reconstruct.

Unlike existing schemes that avoid server interaction, our PRC scheme already requires intra-server communication for aggregation and signing. This allows us to leverage MPC to support AND/multiplication on secret-shared values. Instead of representing the database as a 1-dimensional array of size N , we follow Chor et al. [25] and structure it as a 2-dimensional table of size $(\sqrt{N}, \sqrt{N})$. Clients send two OHE selectors for the target record's row and column, reducing retrieval to a two-layer vector-matrix multiplication. Since servers know the plaintext database, the first layer is computed locally without communication. The second layer, where both vectors are secret shared, has a multiplicative depth of 1 and sublinear server-server communication of $O(\sqrt{N})$.

Existing multi-server PIR schemes either use large non-polylog queries [25, 33, 46] to encode the OHE selector, or rely on a Distributed Point Function (DPF) [43], which prevents computing an arithmetic sharing of the index. Because we employ MPC, clients can instead send Boolean shares of their index, which we expand into an OHE vector on the server side. To support this, we design an efficient conversion from Boolean shares to OHE with multiplicative depth $O(\lg \lg N)$, incurring a server-server communication cost of $1.5\sqrt{N}$ to construct an OHE vector of size $\sqrt{N}$. The low depth of our subprotocols allows us to use information-theoretic MPC protocols, such as GMW [44], which avoid online costs dependent on a symmetric security parameter, while achieving sub-second latency even in WAN settings. Because the majority of our computation is based on Boolean secret sharing, we can efficiently pack eight status bits per byte and apply AVX2 vectorized bitwise operations. This use of SIMD enables a high degree of parallelism while running on a single core, allowing us to achieve a low PRC server computation cost.

2 Related Work

Anonymous credential revocation. A prominent revocation approach for anonymous credentials relies on dynamic accumulators [5, 18, 21, 68], which allow users to prove (non-)membership in an accumulator representing the set of all revoked or valid credentials. An inherent drawback is that each accumulator update (e.g., upon revocation) invalidates existing witnesses, forcing users to either update their witness for every revocation or query for a fresh one.

Blacklist-based revocation methods [14, 81] suffer from expensive non-membership proofs. A possible optimization is requiring the authority to maintain a sorted signature list of revoked identifiers, enabling users to prove that their identifier lies between two adjacent signatures [19, 67]. Yet, constructing, updating, and distributing such lists remains costly. One way to reduce this update overhead is by restricting updates to specific time intervals/epochs; however, this sacrifices the

timeliness of the revocation. In [16, 74], users can present a limited number of unlinkable non-revocation tokens per epoch without expensive proofs or communication with the RA. However, their approach requires an expensive generation of a blacklist of all revoked tokens for the RA and high costs of non-inclusion checks for verifiers. Furthermore, unlinkability does not hold if the verifier colludes with the RA.

Distributing the RA is a prominent approach to avoid single points of failure [45, 50, 53]. Our scheme relies on a *distributed RA* under an anytrust assumption, where the user's privacy is guaranteed if at least one out of n RAs does not collude with the adversary. Clients do not need to know which RA is not compromised. We compare our scheme with the recent multi-party revocation scheme ALLOSAUR [53], which is based on accumulators under an anytrust assumption. ALLOSAUR is not designed for deployments at continental scale, and we show in §7.2 that we improve revocation cost by a factor of $20,000\times$ for 10M credentials and a 1% revocation rate.

Private information retrieval. PIR allows a client to retrieve an element from a database without revealing the queried index to the server. PIR schemes fall into two categories: multi-server PIR, which relies on multiple non-colluding servers [25, 27, 33, 35, 47, 66, 69], and single-server PIR, which uses asymmetric cryptography [31, 52, 59, 63, 69]. In multi-server PIR approaches, the introduction of schemes based on distributed point functions and function secret sharing has enabled sublinear communication [11, 12, 43, 57]. Recent years have seen significant progress in single-server PIR, largely due to advances in homomorphic encryption. Early works such as XPIR [62] and SealPIR [2] demonstrated query compression over RLWE (e.g., [13]), achieving sublinear communication. Later schemes, including SimplePIR [52] and FrodoPIR [31], eliminated the ring structure and achieved throughput comparable to multi-server PIR by operating directly on LWE [73], but required clients to download and store large initialization *client-hints*. Later works [51, 59, 65] removed this requirement at the cost of higher per-query overhead. Another line of research [15, 41, 64], combines RLWE with GSW [42] to represent queries in high dimensions, achieving polylogarithmic communication at the expense of heavy computation and large per-client evaluation keys (or *server-hints*).

Despite these advances, many of the PIR techniques are incompatible with our setting. Real-time credential revocation involves frequent database updates, making approaches with expensive, data-dependent client hints impractical, as their costs cannot be amortized across many queries. Similarly, server-side hints such as per-client evaluation keys are infeasible, since reusing the same key would break the anonymity required for anonymous credentials.

While PIR and PRC share similar settings and construction mechanisms, they have orthogonal goals. In §5.2, we highlight these differences and further provide a naive revocation approach based on PIR. We evaluate this approach

using state-of-the-art multi-server PIR and single-server PIR, showing that beyond protecting database privacy, our construction reduces server cost by a factor of $2700\times$ compared to multi-server PIR and $900\times$ compared to single-server PIR.

Committed OT/authenticated MPC. Committed oblivious transfer [28,29,40] is an OT protocol, in which the participants commit to their inputs. Unfortunately, committed OT incurs a cost linear in the number of records. Similarly, authenticated MPC [34] provides a way for participants to authenticate their inputs to an MPC protocol, but these schemes still require communication linear in the input size.

Blind signatures. Similar to PRC, blind signatures [24,72] let client obtain signatures on a message, without revealing it to the signer. In contrast, in our revocation and PRC setting, the signer must attest to the validity of a data record and not on a freely chosen message. This enforces a specific structure: clients only choose a record index, not its associated content.

Self-sovereign identity. Self-sovereign identity (SSI) is an architectural model for user-centric identity systems allowing users to control and present their credentials without relying on central identity providers [56]. SSI is commonly instantiated using decentralized identifiers and verifiable credentials [61,77–79]. Anonymous credentials can be viewed as verifiable credentials that additionally provide anonymity and unlinkability, and SSI systems can be instantiated with them. Revocation mechanisms for non-anonymous credentials, such as the W3C Bitstring Status List [60], rely on credential identifiers as part of the revocation status check, which is incompatible with anonymous credentials where no such identifier can be disclosed to preserve anonymity. We focus on revocation for anonymous credentials; instantiating SSI systems with them is outside the scope of this work.

3 Preliminaries

In this section, we present our notation, preliminaries, and building blocks used in our construction.

Notation. We use $[n]$ to denote the set $\{0, \dots, n-1\}$ and use $(a_i)_{i \in [n]}$ to represent a list of n elements $(a_0, \dots, a_{n-1})$. When size n is known from the context, we omit it. We denote vectors by bold-face letters like $\mathbf{v}$ and refer to their i -th element as $\mathbf{v}[i]$. We denote inner products by $\langle \mathbf{v}, \mathbf{w} \rangle$ and when both vectors are k -bit arrays compute it as $p = \bigoplus_{i \in [k]} (\mathbf{v}[i] \wedge \mathbf{w}[i])$. We abuse the notation and still use $\langle \mathbf{v}, \mathbf{w} \rangle$ when one of the vectors contains l -bit values. We use $x[a : b]$ to denote the bit slice of $x \in \mathbb{Z}_{2^k}$ from bit position a (inclusive) to b (exclusive), where $0 \leq a \leq b \leq k$. The least significant bit is $x[0 : 1]$, and we omit the index when the slice starts at 0 or ends at k . We use $y \xleftarrow{\$} S$ to denote that y is sampled uniformly at random from a set S . We write $y \leftarrow A(x; r)$ to denote that y is assigned the output of algorithm A running on input x and randomness r . We assume that the security parameter λ and public parameters pp , typically containing group parameters, are implicitly given as

input to all algorithms. A polynomial-time verifiable binary relation is denoted as $(x, w) \in R$, where w is a witness for the statement x satisfying the relation.

One-hot encoding. A One-Hot Encoding (OHE) of an integer x for dimension n , denoted by $\mathbf{v} \leftarrow \text{OHE}(x, n)$, is a bit vector of length n such that $\mathbf{v}[x] = 1$ and $\forall i \neq x, \mathbf{v}[i] = 0$.

Homomorphic commitments. A homomorphic commitment scheme Com consists of the following algorithms:

- $\text{co} \leftarrow \text{Commit}(x; r)$: Generates a commitment co to a value x using randomness r .
- $\text{co} \leftarrow \text{Agg}((\text{co}_i)_{i \in [n]})$: Aggregates the commitments co_i for $i \in [n]$ into a single commitment co .
- $0/1 \leftarrow \text{Open}(\text{co}, x, r)$: Verifies whether the commitment co opens to the value x using randomness r .

Homomorphic commitment schemes must satisfy *binding* and *hiding*. Informally, binding ensures that a commitment co can only be opened to the committed value x with the randomness r and hiding ensures that co does not reveal any information about the committed value x . The homomorphism ensures that $\text{Agg}(\text{Commit}(x_1; r_1), \text{Commit}(x_2; r_2)) = \text{Commit}(x_1 + x_2; r_1 + r_2)$. We will instantiate this homomorphic commitment scheme with Pedersen commitments [70].

(Threshold) signature schemes. We consider both standard and threshold signature schemes, where a standard signature scheme Sig consists of the following algorithms:

- $(\text{sk}, \text{vk}) \leftarrow \text{KGen}(1^\lambda)$: Generates a signing key sk and a verification key vk .
- $\sigma \leftarrow \text{Sign}(\text{sk}, m)$: Generates a signature σ for a message m using the signing key sk .
- $0/1 \leftarrow \text{Verify}(\text{vk}, \sigma, m)$: Verifies the signature σ for a message m using the verification key vk .

A threshold signature scheme TSig in an t -out-of- n setting modifies key generation and signing while keeping compatibility with the standard verification as follows:

- $((\text{sk}_i)_{i \in [n]}, \text{vk}) \leftarrow \text{KGen}(1^\lambda, n)$: Generates a set of n signing keys sk_i and a common verification key vk . This algorithm might also be defined in a distributed manner.
- $\sigma_i \leftarrow \text{PSign}(\text{sk}_i, m)$: Generates a partial signature σ_i for a message m using the signing key sk_i .
- $\sigma \leftarrow \text{Agg}((\sigma_i)_{i \in [n]})$: Aggregates t partial signatures (σ_i) into a single signature σ .

A (threshold) signature scheme needs to satisfy *correctness* and *unforgeability*. Correctness states that a valid signature σ on some message m can be verified using the verification key vk . Unforgeability ensures that no adversary (knowing at most $t-1$ signing keys sk_i in the threshold setting) can generate a valid, new signature σ on some message m . We instantiate the scheme with threshold Schnorr signatures [76].

Zero-knowledge proof systems. A zero-knowledge proof system ZKP allows a prover to generate a proof π for a statement x that convinces a verifier that the prover knows a witness w satisfying a public relation R such that $(x, w) \in R$ with-

out revealing any information about the prover's witness w .

A zero-knowledge proof system should satisfy *completeness*, *knowledge-soundness*, and *zero-knowledge*. Completeness states that honestly generated proofs will always pass verification. Knowledge-soundness states that it is impossible to generate a valid proof π for a statement x without knowing a valid witness w . Zero-knowledge ensures that verifier learns no information beyond the prover knowing a valid witness w . We instantiate our ZKP system with efficient Σ -protocols [80].

Anonymous credentials. We specify an anonymous credential system AC as comprising the following algorithms:

- $(sk, vk) \leftarrow \text{KGen}(1^\lambda)$: Generates the secret key sk and verification key vk .
- $\sigma_{\text{cred}} \leftarrow \text{Issue}(sk, (a_i)_{i \in [m]})$: Issues a credential σ_{cred} with attributes $(a_i)_{i \in [m]}$ using the secret key sk .
- $pt \leftarrow \text{Present}(\sigma_{\text{cred}}, (a_i)_{i \in [m]}, \text{Rvd})$: Generates a presentation token pt for a credential σ_{cred} while selectively revealing the attributes a_i for $i \in \text{Rvd} \subseteq [m]$.
- $0/1 \leftarrow \text{VerifyPresent}(vk, pt, (a_i)_{i \in \text{Rvd}})$: Verifies the presentation token pt for a set of revealed attributes a_i for $i \in \text{Rvd}$ under the verification key vk .

We require an anonymous credential system to satisfy *correctness*, *unforgeability*, and *unlinkability*. Correctness means that an honestly generated presentation token pt of a valid credential σ_{cred} can be verified using the verification key vk . Unforgeability ensures that no adversary without knowledge of the secret key sk can generate a valid presentation token pt that reveals attributes for which the adversary has not been issued a corresponding credential. Lastly, unlinkability ensures that no adversary controlling the issuer and verifier can tell two presentations pt_1 and pt_2 for the same revealed attribute set Rvd of different credentials apart. This ensures that clients cannot be linked across multiple presentations. We provide formal definitions for these properties in the full version [36].

Secret-sharing. An n -out-of- n secret-sharing scheme SS enables sharing of a value x among n parties such that the parties can compute on the shared value $\llbracket x \rrbracket$. Secret-sharing consists of two primary operations:

- $(\llbracket x \rrbracket_i)_{i \in [n]} \leftarrow \text{SS.Share}(x, n)$: Shares a value x into n secret shares $(\llbracket x \rrbracket_i)_{i \in [n]}$.
- $x \leftarrow \text{SS.Rec}((\llbracket x \rrbracket_i)_{i \in [n]})$: Given n secret shares $(\llbracket x \rrbracket_i)$, reconstructs the value x .

We require the secret-sharing scheme to satisfy two properties: *correctness* and *hiding*. The correctness property ensures that computation performed on shared values yields the same result as if it were performed on the original value x . The hiding property ensures that any group of less than n parties cannot learn any information about the secret value x . Secret-sharing can be instantiated via the GMW protocol [44] and the online phase can be improved via Beaver triples [7].

We use two types of secret-sharing: 1) *Arithmetic* sharing which we denote as $\llbracket x \rrbracket^A \leftarrow \text{SS.Share}^A(x)$ where values are taken from a prime field $x \in \mathbb{Z}_q$ and shared such that

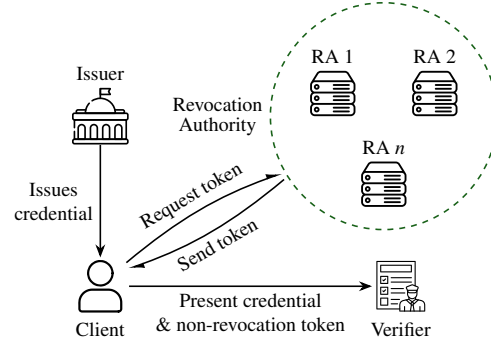


Figure 1: The system model of our anonymous credential revocation system.

$x = \sum_i^n \llbracket x \rrbracket_i \bmod q$. 2) *Boolean* sharing denoted as $\llbracket x \rrbracket^B \leftarrow \text{SS.Share}^B(x)$ where values are k -bit binary values $x \in \mathbb{Z}_{2^k}$ and shared such that $x = \bigoplus_i^n \llbracket x \rrbracket_i^B$ where $\oplus$ is the XOR operation. In addition, we define a conversion operation as follows:

- $\llbracket x \rrbracket^A \leftarrow \text{SS.B2A}(\llbracket x \rrbracket^B)$: Takes Boolean secret shares $\llbracket x \rrbracket^B$ and interactively converts them to arithmetic shares $\llbracket x \rrbracket^A$.

We instantiate this operation via the conversation of Damgård et al. [30], which uses daBits [37, 75] as correlated randomness, and provides a depth-optimized conversion that only requires a single round of interaction.

4 Anonymous Credential Revocation System

This section introduces the system model, requirements, and definitions for anonymous credential revocation.

4.1 System Model

In an anonymous credential revocation system, a client holds a credential and proves its non-revocation status to a verifier. The client first obtains a non-revocation token for its credential from a distributed RA and then presents the token to the verifier together with the credential. For privacy, the RA should not learn for which credential the token was requested.

Conceptually, this approach is similar to the online certificate status protocol (OCSP) with stapling, where web servers obtain a signed OCSP response from an OCSP responder to prove their certificates' validity to a client. In our setting, prior to presenting a credential, a client obtains a non-revocation token from the RA to prove its validity.

The system model (Figure 1) consists of four types of parties: 1) the *issuer*, which issues anonymous credentials to clients; 2) the *client*, which holds a credential and wants to present it while proving that it has not been revoked; 3) the *verifier*, which checks the credential's validity, including non-revocation, before providing a service or granting access; and 4) the *RAs*, which jointly maintain a revocation-status database indicating whether each credential is valid or re-

voked. We consider distributed RAs: n authorities provide the service jointly, and clients or verifiers may interact with them to determine or prove a credential's validity. How the current state of the database is replicated across all RAs is out of scope of our work.

Threat model. We consider adversaries that compromise parties either semi-honestly or maliciously. A *semi-honest* corruption reveals the party's internal state, private values, and view, including all incoming and outgoing messages, but does not allow deviations from the protocol. A *malicious* corruption additionally allows the adversary to deviate from the protocol and to drop or modify the party's messages. For the distributed RAs, we use an any-trust assumption: the adversary may compromise at most $n - 1$ RAs, but clients or verifiers *do not* need to know which RA is honest. We assume communication channels provide confidentiality and integrity (e.g., via TLS), and we do not consider side-channel attacks.

4.2 Requirements & Trust Assumptions

We state the functionality, privacy, and deployment requirements for anonymous credential revocation systems. Next, we explain our trust assumptions for these properties. For formal definitions, we refer to the full version of the paper [36].

Functionality.

RQ.F1: Revocation. Clients can obtain a valid non-revocation token for a credential iff the credential is not revoked, and cannot obtain one for a revoked credential.

RQ.F2: Dynamic Database. The revocation-status database can change at any time, and updates occur frequently.

RQ.F3: Real-time Revocation. Once a credential is revoked, the window in which the holder can still present a non-revocation token must be minimal to prevent abuse. We target validity windows of a few seconds, which is inherently incompatible with settings where both the client and verifier are offline and cannot reach the RAs.

Privacy.

RQ.P1: Client Unlinkability. Adversaries should not be able to link multiple non-revocation presentations of a client.

RQ.P2: Revocation Unlinkability. The RAs should not learn information about the client, including the client's credential validity, revealed attributes, or the verifier to whom the client presents the non-revocation token.

RQ.P3: Database Privacy. The revocation-status database held by the RAs should remain private.

Efficiency.

RQ.D1: Scalability. The system must support nationwide certificate revocation in Europe at a scale exceeding 100M users.

RQ.D2: Lightweight Clients. Clients may run on smartphones with limited resources. Their computation and communication costs must remain small.

RQ.D3: Single Query Efficiency. In on demand scenarios, users typically present a single non-revocation token per interaction. Hence, revocation systems cannot amortize cost over batched queries for one user, and must be efficient for single non-revocation queries.

Trust assumptions. We trust issuers to issue only valid credentials. We trust RAs to revoke a credential iff it is no longer valid, and not to help clients produce valid non-revocation tokens for revoked credentials. In the same way that verifiers trust issuers to honestly issue credentials, they trust RAs to check whether credentials remain valid. We separate issuance from revocation for clarity; in practice, issuers and RAs may be operated by the same parties.

Different properties rely on different adversarial models:

AM.1: Client Unlinkability. The adversary may maliciously compromise the issuer, the verifier, and up to $n - 1$ RAs, and try to link different interactions of the same user.

AM.2: Revocation Unlinkability. The adversary may maliciously compromise the issuer and up to $n - 1$ RAs, and try to learn attributes, the credential's validity, or to which verifier the client presents the credential.

AM.3: Forging Non-Revocation Tokens. The adversary may maliciously compromise a client and semi-honestly compromise up to $n - 1$ RAs, and try to obtain a valid non-revocation token for a revoked credential. Thus, compromised RAs may leak their internal state and views, but do not actively deviate from the protocol to help the client to forge.

Existing non-private revocation systems typically rely on a single RA that is trusted for revocation. Mechanisms such as certificate transparency [58] can provide accountability, but they substantially increase system complexity, and adapting them to a private setting such as ours is non-trivial and costly. We acknowledge that our distributed RA design increases the active-compromise attack surface: unlike a single-RA design, an adversary that actively compromises one of the n RAs may affect revocation correctness.

AM.4: Database Privacy. The adversary may maliciously compromise multiple clients and try to learn information about the revocation-status of non-compromised clients.

4.3 Definition

We provide a formal definition of an anonymous credential system with revocation that fulfills the requirements from §4.2. We denote the credential issuer's keys by (sk_I, vk_I) and those of the RA by $((sk_{\mathcal{R},i})_{i \in [n]}, vk_{\mathcal{R}})$.

Definition 4.1. An anonymous credential system with revocation extends an anonymous credential scheme AC (§3), adopts the KGen and Issue algorithms, and extends presentation and verification to include revocation checks through the following algorithms:

- $((sk_{\mathcal{R},i})_{i \in [n]}, vk_{\mathcal{R}}, \mathbf{D}) \leftarrow \text{RevSetup}(n)$: The setup algorithm generates a secret key $sk_{\mathcal{R},i}$ for each of the n RAs, a common verification key $vk_{\mathcal{R}}$, and initializes the revocation status database $\mathbf{D}$.
- $pt/\perp \leftarrow \text{PresentNonRev}(\sigma_{\text{cred}}, (a_i)_{i \in [m]}, Rvd, vk_{\mathcal{R}})$: Generates a presentation token pt selectively revealing attributes a_i for $i \in Rvd$, and proving non-revocation of credential σ_{cred} under the RA's verification key $vk_{\mathcal{R}}$. If σ_{cred} is revoked, it outputs $\perp$.
- $0/1 \leftarrow \text{VerifyNonRev}(vk_I, vk_{\mathcal{R}}, pt, (a_i)_{i \in Rvd})$: Verifies that a presentation token pt with revealed attributes $(a_i)_{i \in Rvd}$ belongs to a non-revoked credential under the issuer key vk_I and RA key $vk_{\mathcal{R}}$.

AC.Present is a client-local algorithm, but PresentNonRev requires the client to interact with the RA to produce the presentation token, aborting if the credential is revoked. As in classic anonymous credential schemes that assume issuer-verifier collusion, we account for collusion with the RA.

In contrast with issuance, we *do not require* unlinkability between obtaining a non-revocation token and its corresponding presentation session. While issuance contains information about clients, as issuers often see, choose, or attest client attributes, RAs must not learn any information about clients (see RQ.P2) during the revocation check. Moreover, real-time revocation (see RQ.F3) imposes strict timing constraints on revocation tokens, such as requiring a fresh token before each presentation, inherently creating a timing link between token retrieval and presentation.

We require anonymous credential schemes with revocation to satisfy the following five properties: *correctness*, *unforgeability*, *client unlinkability*, *revocation unlinkability* and *database privacy*. We summarize these properties below, state their corresponding trust model and defer formal definitions to the full version [36]. *Correctness* ensures that any honestly generated presentation for a non-revoked credential always verifies, while *unforgeability* ensures that malicious clients cannot produce a valid presentation for a revoked credential even when passively compromising up to $n - 1$ RAs (AM.3). Together, revocation correctness and unforgeability capture the functional requirement of revocation (RQ.F1). *Revocation unlinkability*, based on RQ.P2, focuses solely on RAs and requires that a malicious adversary controlling $n - 1$ RAs (AM.2) cannot distinguish whether two non-revocation sessions originate from the same or different credentials, regardless of their attributes or revocation status. *Credential unlinkability* extends classic credential unlinkability and allows a malicious adversary to compromise $n - 1$ RAs in addition to the issuer and verifier (AM.1) to satisfy RQ.P1; it requires distinguishing the origin of a presentation from two credentials—which are chosen by the adversary—to be hard, as long as both presentations have the same revealed attributes and revocation status, which are shown to the verifier. *Database privacy* prevents malicious clients (AM.4) from learning the revocation

status of credentials they do not own, thus satisfying RQ.P3.

Limitations. Our model relies on an any-trust assumption for the RAs: if all RAs are compromised, they can break client privacy. Moreover, we do not guarantee unforgeability against actively malicious RAs, and actively corrupting a single RA is enough for forging non-revocation tokens, which increases the attack surface. These limitations are not inherent to revocation, but existing solutions that avoid this trust assumption incur orders-of-magnitude higher costs. Our design makes a scalability/trust tradeoff: it supports billions of credentials at the cost of stronger trust assumptions. We note that distributing the RAs does not reduce availability: as long as at least two of the n RAs remain reachable, clients can still obtain non-revocation tokens, and correctness and unforgeability are unaffected by the reduced number of available RAs.

5 Private Record Certification

As the first step toward building a revocation scheme, we define and design Private Record Certification (PRC), a primitive that enables privacy-preserving certification of a data record stored at a specific index in a database.

5.1 Definition

We define a PRC scheme as a protocol between a client and n PRC servers $\mathcal{S}_0, \dots, \mathcal{S}_{n-1}$, with the following sub-protocols:

- $((sk_i)_{i \in [n]}, vk) \leftarrow \text{KGen}(n)$: Generates a secret key sk_i for each PRC server $i \in [n]$, and a verification key vk .
- $((qry_i)_{i \in [n]}, qst) \leftarrow \text{Request}(idx, \sigma_{\text{auth}})$: For an index idx and an optional authorization credential σ_{auth} , the Request algorithm outputs a query qry_i for each PRC server $\mathcal{S}_i$ and a client-local query state qst (e.g. randomness).
- $(\sigma_{\text{rec}}, aux) \leftarrow \text{Response}_{\mathcal{S}}((sk_i, qry_i, \mathbf{D})_{i \in [n]}, vk_I)$: The interactive response protocol is executed between all PRC servers $\mathcal{S} = (\mathcal{S}_i)_{i \in [n]}$, each taking their secret key sk_i , query share qry_i , optionally an authorization verification key vk_I , and the database $\mathbf{D}$ as input. As output, the protocol produces a record certificate σ_{rec} and auxiliary data aux .
- $0/1 \leftarrow \text{Verify}(vk, \sigma_{\text{rec}}, qst, idx, rec, aux)$: For a record certificate σ_{rec} with a query state qst , the verification algorithm checks whether σ_{rec} is valid for the verification key vk , i.e., certifies that $\mathbf{D}[idx] = rec$ with auxiliary data aux .

Properties. We require PRC schemes to satisfy *correctness*, *unforgeability*, *unlinkability*, and optionally *database privacy*. We give an informal description of these properties here and defer formal definitions to the full version [36]. *Correctness* requires that any honest execution yields a valid record certificate. *Unforgeability* ensures that no adversary can produce a valid certificate for a falsified record. In particular, it prevents modifying either the auxiliary data, which may include metadata such as a timestamp, or the record relation, i.e., verifying a certificate for a pair (idx, rec) where the relation

$\mathbf{D}[\text{idx}] = \text{rec}$ does not hold. Unlinkability ensures that the PRC servers learn no information about the clients' identity or record. We model this by requiring that no subset of PRC servers can determine the order of two requests from client's chosen by them. Finally, *database privacy* prevents clients from learning record values for indices for which they do not hold an authorization credential.

Threat Model & Trust Assumptions. PRC is a cryptographic primitive, in which we allow for *malicious* and *semi-honest* corruptions, as defined in §4.1. We use the same threat model as in §4.1. As described above, we consider different adversarial models for different properties. Consider the following models: 1) *Unforgeability*: An adversary maliciously compromises the client and up to $n - 1$ PRC servers are semi-honest (resembling AM.1). 2) *Unlinkability*: An adversary maliciously compromises up to $n - 1$ PRC servers (corresponding to AM.2, AM.3). 3) *DB privacy*: An adversary maliciously compromises the client (similar to AM.4).

5.2 Our PRC Construction at a High Level

We provide a high-level overview of our PRC protocol, but first, we study a naive solution to discuss how our problem differs from PIR and how this impacts the challenges we face.

At first glance, PRC resembles PIR, where a client retrieves a record $\text{rec} = \mathbf{D}[\text{idx}]$ without revealing the index idx to the servers. However, there is a fundamental difference: In PIR, the client learns an unknown value rec , whereas in PRC, the client already knows rec and seeks a certificate attesting a statement about a record held at a private index $\mathbf{D}[\text{idx}]$ to third parties. For example, downloading the entire database satisfies the PIR functionality, but offers no benefit for PRC, where the client aims to prove a statement about the database value at a private index. For our revocation use case, we additionally want certificates to admit efficient proofs-of-knowledge, avoiding the need to reveal idx and rec for verification.

Naive solution. A straightforward approach to construct a PRC protocol is to have the server(s) maintain a secondary database holding signed commitments to each record and its index. For a suitable signature scheme, certificates instantiated this way admit efficient proofs. This reduces the PRC problem to clients retrieving the corresponding signature on a commitment via PIR. Concretely, the commitment database contains $\mathbf{CD}[i] = (\sigma_i, \text{rnd}_i)$, where σ_i is a signature on $(\text{co}_i \parallel \text{aux})$ such that $\text{co}_i = \text{Com.Commit}((i, \mathbf{D}[i]); \text{rnd}_i)$ and aux is auxiliary information, such as a timestamp. However, this naive construction has three major issues: 1) For frequently updated or time-sensitive databases (e.g., using a timestamp or expiration date as auxiliary data), PRC servers must re-sign all records in each time epoch. 2) The records in $\mathbf{CD}$ are often substantially larger than the original records, especially in cases of binary records such as validity status. Each entry includes a signature σ_i (e.g., 64 bytes for Schnorr [76] or 112 bytes for BBS+ [4]) and the commitment randomness rnd_i (32 bytes) resulting in

Algorithm 1 Key generation, request, and verify procedures of PRC.

```

function KGen( $n$ )
1:  $((sk_i)_{i \in [n]}, vk) \leftarrow \text{KGen}(n)$ 
2: return  $((sk_i)_{i \in [n]}, vk)$ 

function Request( $\text{idx}, \sigma_{\text{auth}} = \perp$ )
3:  $\text{rnd} \xleftarrow{\$} \mathbb{Z}_q$ 
4:  $\llbracket \text{rnd} \rrbracket^A \leftarrow \text{SS.Share}^A(\text{rnd})$ 
5:  $\llbracket \text{idx} \rrbracket^B \leftarrow \text{SS.Share}^B(\text{idx})$ 
6: return  $((\text{qry}_i)^n := (\llbracket \text{rnd} \rrbracket_i^A, \llbracket \text{idx} \rrbracket_i^B)^n, \text{qst} := \text{rnd})$ 

function Verify( $vk, \sigma_{\text{rec}}, \text{qst}, \text{idx}, \text{rec}, \text{aux}$ )
7:  $\text{co}_{\text{rec}} \leftarrow \text{Com.Commit}((\text{idx}, \text{rec}); \text{qst})$ 
8: return  $\text{TSig.Verify}(vk, \sigma_{\text{rec}}, \text{co}_{\text{rec}} \parallel \text{aux})$ 

```

an expansion of over $768\times$ in the database size for binary records and a proportional increase in PIR cost, which scales linearly with the database size. 3) Extending the naive PIR solutions to provide database privacy (often called Symmetric PIR or Oblivious Transfer) i.e., preventing clients from querying records they are unauthorized to access, is expensive.

We implement and benchmark this naive approach using state-of-the-art PIR schemes in both single- and multi-server settings in §7. Even ignoring issues (1) and (3), high update costs and lack of database privacy, our proposed solution outperforms this naive baseline by a factor of $1000\times$.

Our high-level approach. To overcome these issues, we construct our PRC protocol such that the PRC servers only hold a database $\mathbf{D}$ of the actual record values and compute record certificates on the fly using a tailored, efficient, secure MPC protocol. At a high level, our PRC scheme works as follows.

The client sends a Boolean secret sharing of their requested index idx and an arithmetic secret sharing of a randomness rnd to the PRC servers $\mathcal{S}_i$. Given these shares, the PRC servers collectively compute the secret sharing of the requested record value $\llbracket \text{rec} = \mathbf{D}[\text{idx}] \rrbracket^A$ combining techniques from PIR and MPC. Next, each $\mathcal{S}_i$ commits to its (arithmetic) share of the index and record value using the randomness share provided by the client as $\text{co}_i \leftarrow \text{Commit}((\llbracket \text{idx} \rrbracket_i^A, \llbracket \text{rec} \rrbracket_i^A); \llbracket \text{rnd} \rrbracket_i^A)$. By ensuring that our arithmetic sharing operates in a prime modulus $\mathbb{Z}_q$ consistent with the group order of our homomorphic commitment scheme Com , aggregating the individual commitments implicitly reconstructs to the index and record value in the commitment $\text{co}_{\text{rec}} \leftarrow \text{Agg}((\text{co}_i)_{i \in [n]}) = \text{Commit}((\text{idx}, \text{rec}); \text{rnd})$. Lastly, the PRC servers compute a threshold signature σ_{rec} on the aggregated commitment co_{rec} and auxiliary information aux . Due to the hiding property of the commitment, the PRC servers do not learn any information about the committed index and record shares of other servers. The client can now prove statements about the relation between their index idx and its corresponding record rec , contained in the same commitment co_{rec} , on which they have obtained a signature σ_{rec} valid under the verification key vk .

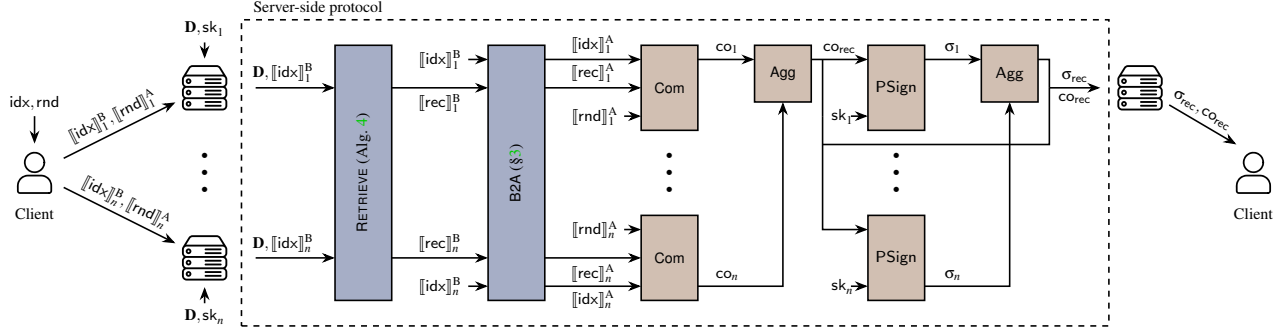


Figure 2: Overview of our PRC construction. Algorithms based on secret sharing are shown in blue, and those using asymmetric operations are shown in brown. For simplicity, auxiliary information aux is omitted.

Algorithm 2 Multiparty response protocol for privately signing a commitment to a database index idx and its record $\text{rec} = \mathbf{D}[\text{idx}]$.

Public server input: Database $\mathbf{D} \in \mathbb{Z}_{2^l}^N$, auxiliary data aux, key vk.
Private input of $\mathcal{S}_i$: Secret key sk_i , query share $q_i = ([\text{rnd}]_i^A, [\text{idx}]_i^B)$.
Output: Tuple $(\sigma_{\text{rec}}, \text{aux})$, where $\text{TSig.Verify}(vk, \sigma_{\text{rec}}, \text{co}_{\text{rec}} \parallel \text{aux})$, for $\text{co}_{\text{rec}} \leftarrow \text{Com.Commit}([\text{idx}], \mathbf{D}[\text{idx}]); \text{rnd})$.
Response protocol:

1. All servers interactively compute $[\text{rec}]^B \leftarrow \text{RETRIEVE}(\mathbf{D}, [\text{idx}]^B)$.
2. All servers interactively convert their binary shares to arithmetic shares: $[\text{idx}]^A \leftarrow \text{SS.B2A}([\text{idx}]^B)$, $[\text{rec}]^A \leftarrow \text{SS.B2A}([\text{rec}]^B)$.
3. Each server $\mathcal{S}_i$ commits to its index and record share as $\text{co}_i \leftarrow \text{Com.Commit}([\text{idx}]_i^A, [\mathbf{D}[\text{idx}]]_i^A); [\text{rnd}]_i^A$ and sends it to $\mathcal{S}_0$.
4. Server $\mathcal{S}_0$ aggregates all commitments: $\text{co}_{\text{rec}} \leftarrow \text{Com.Agg}([\text{co}_i]_{i \in [n]})$. Since the commitment is additively homomorphic and has the same group order as our arithmetic sharing, it holds that: $\text{co}_{\text{rec}} = \text{Com.Commit}([\text{idx} = \sum_i [\text{idx}]_i^A, \text{rec} = \sum_i [\text{rec}]_i^A]; r = \sum_i [\text{rnd}]_i^A)$.
5. Server $\mathcal{S}_0$ broadcasts commitment co_{rec} and auxiliary data aux. Each $\mathcal{S}_i$ computes $\sigma_i \leftarrow \text{TSig.PSign}(sk_i, \text{co}_{\text{rec}} \parallel \text{aux})$ and sends it to $\mathcal{S}_0$.
6. Server $\mathcal{S}_0$ aggregates all signatures as $\sigma_{\text{rec}} \leftarrow \text{TSig.Agg}([\sigma_i]_{i \in [n]})$ and sends signature σ_{rec} and auxiliary data aux to the client.

5.3 Constructing Private Record Certification

We now construct a basic version of PRC, as defined in §5.1, omitting the optional authorization credential σ_{auth} . In §5.4, we then extend this protocol to achieve database privacy. We present PRC algorithms executed by a single party in Alg. 1, and the more complex multi-party Response protocol separately in Alg. 2. The full construction is illustrated in Figure 2. We construct our PRC scheme as follows:

- $((sk_i)_{i \in [n]}, vk) \leftarrow \text{KGen}(n)$: The key generation uses a threshold signature scheme TSig to generate secret keys sk_i for each of the n servers and a public verification key vk.
- $((\text{qry}_i)_{i \in [n]}, \text{qst}) \leftarrow \text{Request}(\text{idx}, \perp)$: The request algorithm samples commitment randomness $\text{rnd} \xleftarrow{\$} \mathbb{Z}_q$ and then secret shares rnd via an arithmetic sharing mod q , where q is the commitment group order, and the index idx via Boolean sharing. The client sends the shares $\text{qry}_i = ([\text{rnd}]_i^A, [\text{idx}]_i^B)$

Algorithm 3 Converting a boolean secret sharing $[\![x]\!]^B$ of a k -bit value $x \in \mathbb{Z}_{2^k}$ to a boolean secret sharing of its one-hot encoding.

```

function Bin2OHE( $k, [\![x]\!]^B$ )
1: if  $k = 1$  then
2:   return  $[\![\neg x]\!]^B, [\![x]\!]^B$ 
3:  $m \leftarrow \lceil k/2 \rceil$ 
4:  $[\![\mathbf{l}]\!]^B \leftarrow \text{Bin2OHE}(m, [\![x[:m]]\!]^B)$ 
5:  $[\![\mathbf{r}]\!]^B \leftarrow \text{Bin2OHE}(k - m, [\![x[m:]]\!]^B)$ 
6: for  $i \leftarrow [2^m]$  do
7:   for  $j \leftarrow [2^{k-m}]$  do
8:      $[\![\mathbf{o}[i + j2^m]]\!]^B = [\![\mathbf{l}[i]]\!]^B \wedge [\![\mathbf{r}[j]]\!]^B$ 
9: return  $[\![\mathbf{o}]\!]^B$ 

```

Algorithm 4 Given a plaintext database $\mathbf{D} \in \mathbb{Z}_{2^l}^N$ of N l -bit records and a boolean secret shared index $[\![\text{idx}]\!]^B$, computes the secret sharing of the requested record $[\![\text{rec}]\!]^B = [\![\mathbf{D}[\text{idx}]]\!]^B$.

```

function RETRIEVE( $\mathbf{D}, [\![\text{idx}]\!]^B$ )
1:  $\eta \leftarrow \lceil \lg(N) \rceil$ 
2:  $m \leftarrow \lceil (\eta + \lg(l))/2 \rceil$ 
3:  $\mathbf{D}'[i][j] \leftarrow \mathbf{D}[i2^m + j]$  ▷ Rearrange  $\mathbf{D}$  as a 2-dim table
4:  $[\![\mathbf{r}]\!]^B \leftarrow \text{Bin2OHE}(m, [\![\text{idx}[:m]]\!]^B)$  ▷ Get row idx share
5:  $[\![\mathbf{c}]\!]^B \leftarrow \text{Bin2OHE}(\eta - m, [\![\text{idx}[m:]]\!]^B)$  ▷ Get column idx share
6: for  $i \leftarrow [2^{\eta-m}]$  do
7:    $[\![\mathbf{a}[i]]\!]^B = \langle \mathbf{D}'[i], [\![\mathbf{r}]\!]^B \rangle$  ▷ Local operation as  $\mathbf{D}'[i]$  is plain
8:  $[\![\text{rec}]\!]^B = \langle [\![\mathbf{a}]\!]^B, [\![\mathbf{c}]\!]^B \rangle$ 
9: return  $[\![\text{rec}]\!]^B$ 

```

to the corresponding server and stores the state $\text{qst} := \text{rnd}$.

- $(\sigma_{\text{rec}}, \text{aux}) \leftarrow \text{Response}_S((sk_i, \text{qry}_i, \mathbf{D})_{i \in [n]}, \perp)$: The following response protocol (Alg. 2) is run among the servers: 1) Given the Boolean-shared index $[\![\text{idx}]\!]^B$ for each PRC server $\mathcal{S}_i$, the PRC servers interactively compute their sharing of the record value as $[\![\text{rec}]\!]^B \leftarrow \text{RETRIEVE}(\mathbf{D}, [\![\text{idx}]\!]^B)$ using Alg. 4. We explain the retrieval algorithm in more detail below. 2) The PRC servers convert the Boolean-shared index $[\![\text{idx}]\!]^B$ and record value $[\![\text{rec}]\!]^B$ to arithmetic shares $[\![\text{idx}]\!]_i^A$ and $[\![\text{rec}]\!]_i^A$ using B2A, see §3. 3) They commit to their arithmetic index and record share using the randomness shares provided by the client as

- $\text{co}_i \leftarrow \text{Com.Commit}(\llbracket \text{idx} \rrbracket_i^A, \llbracket \text{rec} \rrbracket_i^A); \llbracket \text{rnd} \rrbracket_i^A$) and 4) aggregate all commitments as $\text{co}_{\text{rec}} \leftarrow \text{Com.Agg}((\text{co}_i)_{i \in [n]})$. 5) They individually compute partial signatures $\sigma_i \leftarrow \text{TSig.PSign}(\text{sk}_i, \text{co}_{\text{rec}} \parallel \text{aux})$ on the aggregated commitment co_{rec} and auxiliary information aux . 6) Lastly, they aggregate the partial signatures as $\sigma_{\text{rec}} \leftarrow \text{TSig.Agg}((\sigma_i)_{i \in [n]})$, and output $(\sigma_{\text{rec}}, \text{co}_{\text{rec}}, \text{aux})$ to the client.
- $0/1 \leftarrow \text{Verify}(\text{vk}, \sigma_{\text{rec}}, \text{qst}, \text{idx}, \text{rec}, \text{aux})$: The verification algorithm computes $\text{co}_{\text{rec}} \leftarrow \text{Com.Commit}(\llbracket \text{idx}, \text{rec} \rrbracket; \text{qst})$ and verifies that $\text{TSig.Verify}(\text{vk}, \sigma_{\text{rec}}, \text{co}_{\text{rec}} \parallel \text{aux})$.

Private retrieval. Now we explain how given a database $\mathbf{D}$ of N l -bit records and a Boolean-sharing of index $\llbracket \text{idx} \rrbracket^B$, PRC servers compute a secret sharing of record $\llbracket \text{rec} \rrbracket^B = \llbracket \mathbf{D}[\text{idx}] \rrbracket^B$ via our RETRIEVE algorithm given in Alg. 4. As we have multiple PRC servers, we base our construction on multi-server PIR protocols [25, 33, 46]. In classical multi-server PIR schemes, the client creates a secret-shared OHE of its index, which is of database size N . It then sends the share to the PRC servers, who non-interactively compute their response. We avoid this load on clients by allowing them to send Boolean shares of their index to the PRC servers, who then collaboratively compute the OHE. This expansion requires interaction, which is acceptable in our case as PRC servers already interact for the aggregation and threshold signature. This enables us to achieve a minimal client-side communication and computation cost of $\lg N$.

We start by building a conversion $\text{BIN2OHE}(k, \llbracket x \rrbracket^B)$ that takes a k -bit Boolean secret shared index x and recursively computes a sharing of $\llbracket \text{OHE}(k, x) \rrbracket^B$, as shown in Alg. 3. Having a 1-bit shared index $\llbracket x \rrbracket^B$, building the OHE vector, which has two elements, is trivial as $\llbracket \text{OHE}(1, \llbracket x \rrbracket^B) \rrbracket^B = [-\llbracket x \rrbracket^B, \llbracket x \rrbracket^B]$. To build $\llbracket \text{OHE}(k, \llbracket x \rrbracket^B) \rrbracket^B$ for a k -bit index x , we treat x as vector $\llbracket \mathbf{x}_{i \in [k]} \rrbracket^B$ of k secret shared bits and split the shared bits of index into lower $\llbracket \mathbf{x}[:k/2] \rrbracket^B$ and upper $\llbracket \mathbf{x}[k/2:] \rrbracket^B$ halves, then recursively compute their OHEs. Finally, we build an OHE vector of size 2^k , where each bit is computed by a single AND operation, for each combination of elements between the lower and upper OHE vectors.

Now that we can convert secret shared indices to OHEs, we can apply the conversion to the client-shared index and perform a classic XOR PIR. However, expanding the index to a vector of size N requires communication linear in the number of records. Instead, we follow Chor’s guideline [25] and rearrange the database as a two-dimensional table. This enables us to split the OHE index generation into two parts of size approximately $\sqrt{N}$, supporting sublinear communication cost for the OHE expansion: given an index $\text{idx} = c2^m + r$ represented as $(\llbracket c \rrbracket^B, \llbracket r \rrbracket^B)$, we compute two OHE vectors $\llbracket \mathbf{c} \rrbracket^B = \text{BIN2OHE}(\eta - m, c)$ and $\llbracket \mathbf{r} \rrbracket^B = \text{BIN2OHE}(m, \llbracket r \rrbracket^B)$ where $\eta = \lg(N)$. The first vector $\llbracket \mathbf{c} \rrbracket^B$ corresponds to the column index of the record in the database, while the second vector $\llbracket \mathbf{r} \rrbracket^B$ corresponds to the row index. By performing an XOR PIR via the row index, through $\llbracket \mathbf{a}[i] \rrbracket^B = \langle \mathbf{D}'[\mathbf{i}], \llbracket \mathbf{r} \rrbracket^B \rangle$,

Table 1: The number of AND gates, $C(k)$, used for a k -bit $\text{BIN2OHE}(k, x)$ conversion. We denote overhead over the output size 2^k in parenthesis.

i	$C(i)$	i	$C(i)$	i	$C(i)$
1	0	7	164 (+28%)	13	8444 (+3.1%)
2	4 (0)	8	304 (+19%)	14	16712 (+2%)
3	12 (+50%)	9	584 (+14%)	15	33236 (+1.4%)
4	24 (+50%)	10	1120 (+9.8%)	16	66144 (+0.92%)
5	48 (+50%)	11	2184 (+6.6%)	17	131960 (+0.68%)
6	88 (+38%)	12	4272 (+4.3%)	18	263312 (+0.45%)

we can select the desired row in the database and reduce the dimension to one. As the PRC servers hold the plaintext database $\mathbf{D}$, this operation only requires performing AND with known values, which can be done locally by each $\mathcal{S}_i$. Next, the PRC servers interact to compute the inner product of the resulting vector $\llbracket \mathbf{a}[i] \rrbracket^B$ with the column index to retrieve the desired record value $\llbracket \text{rec} \rrbracket^B = \langle \llbracket \mathbf{a} \rrbracket^B, \llbracket \mathbf{c} \rrbracket^B \rangle$.

PRC assumes that clients know the record value. If this assumption does not hold, clients can either brute-force the value by verifying the token against the record domain (e.g., valid or revoked for a revocation status), or PRC servers can attach encrypted shares of the record $\llbracket \text{rec} \rrbracket^B$ to the response, effectively yielding a PIR query.

Performance. We analyze the computation and communication costs of our RETRIEVE algorithm for PRC servers next.

First, we analyze the cost of BIN2OHE in Alg. 3 using induction. Let $C(k)$ denote the number of AND gates required to compute $\text{BIN2OHE}(k, x)$, a k -bit conversion, with base case $C(1) = 0$. In lines 4–5, we recursively call the conversion, halving the index’s input size, and incurring costs of $C(\lceil k/2 \rceil)$ and $C(\lfloor k/2 \rfloor)$, while the loop on lines 6–8 incurs a fixed cost of 2^k AND gates. Combined, this results in a cost of $C(k) = C(\lceil k/2 \rceil) + C(\lfloor k/2 \rfloor) + 2^k$ AND gates. We report the concrete cost of converting a k -bit index to its OHE representation and the resulting overhead relative to the 2^k -sized output vector in Table 1. The cost is bounded by $C(k) \leq 1.5 \cdot 2^k$ ANDs gates, but this overhead diminishes rapidly with increasing k .

The RETRIEVE algorithm in Alg. 4 calls BIN2OHE twice on lines 4–5, incurring communication and computation costs of $1.5(2^{\eta-m} + 2^m)$. The loop on line 6 performs an inner product between a secret-shared row index vector and the plaintext database $\mathbf{D}$. As the database is known, this computation can be performed locally leading to a computation cost $O(2^{\eta-m} \cdot l2^m = lN)$ and *no communication cost*. The final, inner product on line 8 is between two secret shared vectors of size $2^{\eta-m}$ but gets repeated for each bit of our l -bit record, leading to a computation and communication cost of $l \cdot 2^{\eta-m}$. This leads to a total computation cost of $O(lN)$, as required by the lower bound of private retrieval [48], and a communication cost of $O(2^m + (l+1) \cdot 2^{\eta-m})$. We set m in an optimized manner such that $2^m \approx l \cdot 2^{\eta-m}$, which leads to a total communication cost

of $O(\sqrt{IN})$ that is sublinear in the size of the database.

We analyze the multiplicative depth of our protocol (i.e., the minimal number of sequential AND layers) as it determines the round complexity and affects system latency. Our BIN2OHE conversion has a depth of $\lg k$, as it runs two instances of size $k/2$ in parallel, then performs one AND layer. Our RETRIEVE algorithm calls two BIN2OHE conversions in parallel, then runs one local (line 7) and one interactive (line 8) AND layer to compute inner products. This results in an overall depth of $1 + \lg \lg N$ as $k = \lg N$ in BIN2OHE. This low depth lets us use high-latency communication-efficient MPC protocols like GMW [44] while still achieving real-time performance (§7).

Using a threshold signature. We use threshold signatures even though our PRC construction does not protect against malicious PRC servers forging certificates, as they can modify $\llbracket \text{rec} \rrbracket^B$ in our semi-honest MPC protocol. The threshold design mainly improves availability, enables load-balancing aggregation, and simplifies verifier key management when the set of active PRC servers changes. Nevertheless, threshold signatures can be trivially replaced with other signature schemes.

5.4 Extension for Database Privacy

We extend our basic construction to support database privacy by restricting client queries to records for which they hold a valid authorization credential σ_{auth} . The authorization credential can be instantiated with a signature or anonymous credential, as long as it is unforgeable and allows for efficient zero-knowledge proofs of knowledge. To verify σ_{auth} , we assume its verification key vk_I to be known to all parties. We outline the changes in our PRC construction next.

- $((\text{qry}_i)_{i \in [n]}, \text{qst}) \leftarrow \text{Request}(\text{idx}, \sigma_{\text{auth}})$: The client holds a certificate σ_{auth} on its index idx . The client holds a certificate σ_{auth} on its index idx . In addition to performing the basic Request in Alg. 1, it creates a zero-knowledge proof π_{auth} for relation R_{auth} (see §A). Specifically, it samples extra randomness $\text{rnd}_{\text{idx}} \xleftarrow{\$} \mathbb{Z}_q$ and secret shares for all the PRC servers, enabling to produce a second commitment on the index $\text{co}_{\text{idx}} \leftarrow \text{Com.Commit}(\text{idx}; \text{rnd}_{\text{idx}})$ similar to co_{rec} . The client pre-computes this commitment co_{idx} and computes the zero-knowledge proof π_{auth} proving knowledge of a certificate σ_{auth} on the same idx as in co_{idx} . It extends each query as $\text{qry}_i = (\llbracket \text{rnd} \rrbracket_i^A, \llbracket \text{rnd}_{\text{idx}} \rrbracket_i^A, \llbracket \text{idx} \rrbracket_i^B, \pi_{\text{auth}})$.
- $(\sigma_{\text{rec}}, \text{aux}) \leftarrow \text{Response}_S((\text{sk}_i, \text{qry}_i, \mathbf{D})_{i \in [n]}, \text{vk}_I)$: In a similar process to co_{rec} in Alg. 2, the S_i compute $\text{co}_{\text{idx}} \leftarrow \text{Com.Commit}(\sum \llbracket \text{idx} \rrbracket_i^A; \sum \llbracket \text{rnd}_{\text{idx}} \rrbracket_i^A)$. Then, they verify the zero-knowledge proof π_{auth} using co_{idx} and vk_I , abort if verification fails, and send the final response to the client.

This modification does not impact the security of the protocol, which we study next. In particular, the zero-knowledge property of the proof system ensures that no information about the index idx is revealed, thus preserving unlinkability.

5.5 Security of our PRC Construction

We present proof sketches for our PRC construction here, and defer formal proofs to the full version of the paper [36].

Informal Theorem 5.1 (Unforgeability). *Our PRC construction achieves unforgeability if the threshold signature scheme TSig is unforgeable, and the commitment scheme Com is binding.*

For unforgeability, we consider a malicious client and up to $n - 1$ semi-honest PRC servers. Since all PRC servers follow the protocol, the client can only obtain signatures on commitments to an index and its corresponding record value and auxiliary data. A certificate forgery thus either requires opening a valid certificate to different content (commitment binding) or forging a signature.

Informal Theorem 5.2 (Unlinkability). *Our PRC construction achieves unlinkability if the commitment scheme Com is hiding and the correlated randomness is private.*

We require the correlated randomness to be private, i.e., randomness shares of honest parties are uniform and unknown to the adversary. Our security argument is similar to the approach of Araki et. al. [3] for malicious privacy. Given private correlated randomness, we obtain protection against malicious PRC servers even though we use a semi-honest MPC protocol [44]. An adversary may tamper with shares, breaking integrity and introducing additive errors; however, since no output value is reconstructed, these errors remain hidden and leak no information. In more detail, our construction consists of two components: 1) a semi-honest MPC protocol computing $(\llbracket \text{idx} \rrbracket_i^A, \llbracket \mathbf{D}[\text{idx}] \rrbracket_i^A)$ and 2) a custom-made protocol using asymmetric operations (commitment and signature). At the end of the MPC computation, each party commits to its share using the client-chosen randomness rnd_i and publishes the commitment co_i . Assume that S_h is the non-colluding PRC server. By the hiding property of the commitment, co_h reveals no information about its content $(\llbracket \text{idx} \rrbracket_h^A, \llbracket \text{rec} \rrbracket_h^A; \llbracket \text{rnd} \rrbracket_h^A)$. We assume that the commitment aggregation is adversarial. If the aggregation includes co_h , then the randomness rnd_h protects the output co_{rec} even if the adversary tampers with co_h . Otherwise, the resulting aggregation contains no share from S_h , and is therefore independent of the client's input by the security of the secret sharing scheme.

Informal Theorem 5.3 (Database Privacy). *Our extended protocol (§5.4) achieves database privacy if the authorization credential is unforgeable and the zero-knowledge proof system is knowledge-sound.*

As all PRC servers are honest, the protocol response only contains information about the requested index. By the knowledge-soundness of the proof system, clients can only generate valid proofs if they know an authorization credential. Further, if the authorization credentials are unforgeable, clients learn nothing about unauthorized indices.

6 Anonymous Credential Revocation via PRC

We construct an anonymous credential revocation system, as defined in §4.3, based on our PRC scheme from §5.2 and an anonymous credential scheme that supports efficient zero-knowledge proofs. Our system includes an issuer I that issues credentials and a decentralized RA $\mathcal{R}$ of n PRC servers that manage a binary revocation status database $\mathbf{D}$. W.l.o.g., we assume that the issuer sets the first attribute a_0 of each credential to the corresponding revocation index idx and a credential is valid if $\mathbf{D}[\text{idx}] = 1$. As our system is focused on revocation, we omit the issuer in the rest of the section. Our construction works as follows:

- $((\text{sk}_{\mathcal{R},i})_{i \in [n]}, \text{vk}_{\mathcal{R}}, \mathbf{D}) \leftarrow \text{RevSetup}(n)$: Initialize a status database $\mathbf{D}$ and a PRC instance $((\text{sk}_{\mathcal{R},i})_{i \in [n]}, \text{vk}_{\mathcal{R}}) \leftarrow \text{PRC.KGen}(n)$, which produces the verification key $\text{vk}_{\mathcal{R}}$.
- $\text{pt}/\perp \leftarrow \text{PresentNonRev}(\sigma_{\text{cred}}, (a_i)_{i \in [m]}, \text{Rvd}, \text{vk}_{\mathcal{R}})$: The client generates a presentation including a non-revocation proof for a credential σ_{cred} with index $\text{idx} = a_0$ as follows: 1) the client performs a PRC request for idx with the RAs, who set $\text{aux} \leftarrow \text{time}$ as a timestamp. It receives $(\sigma_{\text{rec}}, \text{time})$ and stores the query state qst . 2) the client checks if $\text{PRC.Verify}(\text{vk}_{\mathcal{R}}, (\sigma_{\text{rec}}, \text{qst}), \text{idx} = a_0, \text{rec} = 1, \text{time})$, returning $\perp$ if the check fails. 3) the client computes a zero-knowledge proof π_{rev} for relation R_{rev} (see §A), proving knowledge of a valid credential σ_{cred} with revealed attributes $(a_i)_{i \in R_{\text{vd}}}$, where the PRC commitment co_{rec} opens to a valid status ($\text{rec} = 1$) and the same index idx as the hidden attribute a_0 . 4) the client outputs $\text{pt} := (\sigma_{\text{rec}}, \text{co}_{\text{rec}}, \text{time}, \pi_{\text{rev}})$.
- $0/1 \leftarrow \text{VerifyNonRev}(\text{vk}_I, \text{vk}_{\mathcal{R}}, \text{pt}, (a_i)_{i \in R_{\text{vd}}})$: The verifier parses $\text{pt} := (\sigma_{\text{rec}}, \text{co}_{\text{rec}}, \text{time}, \pi_{\text{rev}})$ and checks 1) if the timestamp is valid and within an application-specific limit, 2) if $\text{TSig.Verify}(\text{vk}_{\mathcal{R}}, \sigma_{\text{rec}}, \text{co}_{\text{rec}} || \text{time})$, i.e., the PRC certificate is valid, and 3) if proof π_{rev} , which links the credential presentation to the non-revocation token, is valid.

Our revocation scheme can be extended to achieve database privacy and satisfy RQ.P3 by using σ_{cred} as an authorization credential in the extended PRC construction (§5.4). We also support multi-show revocation tokens, by employing a ZK-friendly or re-randomizable signature in the PRC construction. We discuss these extensions in more detail in §B.

Security analysis. Next, we provide an intuition why our construction fulfills the security properties defined in §4.3. Other credential properties of interest, such as device-binding, are inherited by the underlying credential scheme and out of scope of our discussion. Our construction links a PRC non-revocation token to a credential presentation with a zero-knowledge proof, ensuring that both components use the same index idx without revealing it. Given the completeness and knowledge-soundness of our proof system, revocation correctness and unforgeability follow directly from the properties of PRC and the underlying credential scheme. In addition,

database privacy trivially follows from PRC. Revocation unlinkability follows from the unlinkability of the PRC scheme, since the adversary only observes the non-revocation token retrieval and not the credential presentation, which an honest verifier obtains. Client unlinkability requires the adversary to choose challenge credentials that have the same revealed attributes and revocation status, leading to the following two cases: 1) If both credentials are revoked, the presentations abort after retrieving an invalid non-revocation token, and unlinkability follows from PRC. 2) If both credentials are valid, the client retrieves a non-revocation token with a fixed status $\text{rec} = 1$ and presents it alongside their credential. While the adversary knows the one-to-one mapping between retrieval and presentation sessions, unlinkability follows from combining the unlinkability of the PRC and the credential scheme.

Instantiation. We can efficiently instantiate our PRC scheme with a simple signature scheme like Schnorr [76], or threshold variants like [9, 26, 55], and the Pedersen commitment scheme [70]. The anonymous credential scheme may be instantiated with BBS/BBS+ [17, 80], or PS [71] credentials. These schemes support efficient zero-knowledge proofs, proving the credential and commitment both contain the same index, via a standard Σ -proof of discrete logarithm equality.

7 Implementation & Evaluation

We implement a prototype of our construction from §5.3, evaluate its performance, and compare the cost of our credential revocation against the state-of-the-art revocation schemes and the naive PIR-based baseline in §5.2.

7.1 Implementation and Setup

We provide an open-source implementation of our PRC construction for single-bit records in Rust¹. We rely on AVX2 to massively parallelize our operations and pack eight status bits in each single byte. AVX2 enables vectorized bitwise operations over 256-bit registers, supporting up to 256 logical operations per cycle while running on a single CPU core.

Experimental setup. We run the experiments on a machine with an AMD Ryzen AI 7 PRO 350 and 64 GB of RAM. We emulate running n PRC servers on a single machine. We rely on TCP for communication in our prototype, while using the ‘tc’ command to shape the network and set the round-trip time (RTT). All experiments are performed in a single core setting, and we use User+System time to report the computation time.

Communication channel security. Our prototype omits TLS management and certificate handling, and uses TCP for simplicity; a real deployment, however, must use TLS or an equivalent secure channel. A single TLS handshake already exceeds the client’s total communication cost for an entire PRC

¹<https://zenodo.org/records/20433788>

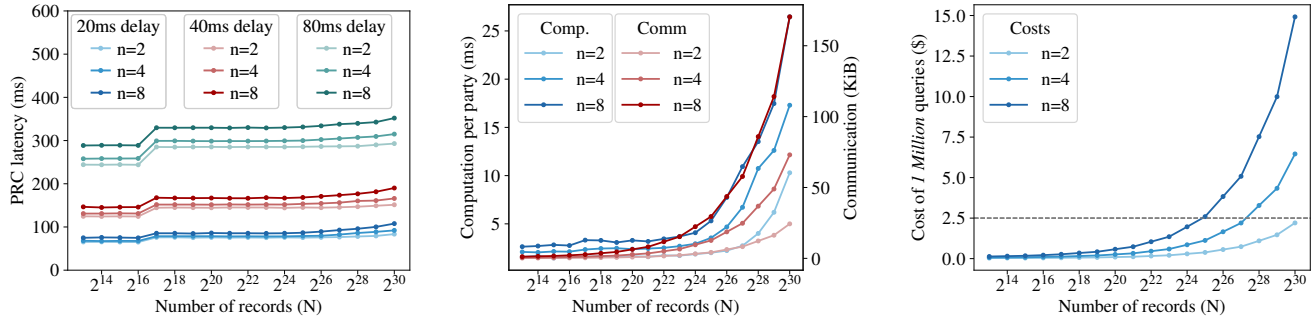


Figure 3: Evaluating the PRC latency (left), *single-core* computation and one-way communication per PRC server (middle), and the operational cost per PRC server (right) over various number of records in database (N) and number of PRC servers (n).

execution, highlighting the communication efficiency of our client-side protocol. In contrast, PRC servers can maintain persistent communication channels among themselves because they handle many requests, and are therefore not impacted by the cost of initiating secure connections. Client-side TLS overhead can be reduced by opening a single TLS connection, onion-encrypting the queries for the other PRC servers, and letting one server relay them over persistent connections, similarly to our offline-client extension (see §B).

Instantiation. We instantiate our MPC using the GMW protocol [44], our homomorphic commitment with the Pedersen scheme over the prime-order Ristretto group [32, 49], and our signatures with Ed25519 [8, 54]. While GMW achieves highly efficient communication without the linear security parameter expansion common in garbled circuits [82], it requires one interaction round per layer of AND gates, often resulting in high latency since typical circuits have thousands of layers. By designing tailor-made protocols with depth $O(\lg \lg N)$ (see §5.3), we achieve both low communication and low latency.

Preprocessing. Efficient SMC protocols heavily rely on preprocessing to reduce their latency and online phase cost. We use Beaver triples [7] for performing AND operations and daBits [37, 75] for B2A conversions. The majority of MPC protocols have expensive preprocessing phases requiring GiBs of communication per execution, however, our protocol only requires sub-linear preprocessing material; specifically, $\sim 3\sqrt{N}$ Beaver triples and $\lg(N) + l$ dabits requiring a transfer of 13.5 KiB for all PRC servers combined when generated via a dealer in our largest setting with $N = 1$ billion credentials. Since our preprocessing is generic and function-independent, we can generate material in large batches and use it across many requests, for lower amortized costs. Correlation generation heavily depends on the deployment scenario, as the cost and threat model often differ between preprocessing and online phases. Benchmarking generic pre-processing generation is out of scope of this work. We use a dealer with pre-shared keys and discuss costs and alternatives in the appendix §E.

7.2 Performance

We evaluate the performance of our PRC and revocation system, comparing it with the baseline and existing solutions.

Latency. Our system is designed to achieve low latency, and our MPC computation only requires six rounds of interactions. Most of this round complexity arises from the Boolean-sharing to one-hot-encoding conversion (Alg. 3), which has depth $\lg \lg(N)$ and requires four rounds for databases with $N = 1$ billion records. The retrieval step (Alg. 4) requires a single round to compute the second inner product. We further adopt a depth-optimized, single-round, Boolean to arithmetic conversion based on [30] using DaBit [75]. We evaluate the latency of our PRC from receiving a query to generating the token σ_{rec} . The left plot in Figure 3 shows system latency as the number of PRC servers varies, with RTTs ranging from 20 ms to 80 ms. Our solution achieves *real-time* responses with latency below 400 ms. The plot also indicates that latency is dominated by the network delay rather than the number of credentials, servers, or computation cost. We observe a step increase on all curves after 2^{16} records, consistent with our theoretical analysis, as BIN2OHE requires an additional round once the index size exceeds 2^8 bits.

Cost. We measure the computation and communication cost of our scheme for different numbers of PRC servers and database records and report the results in the middle plot of Figure 3. We observe a constant cost of ~ 1 ms for small databases, beyond which the cost grows linearly with the number of records. Using vectorized AVX2 operations, our system achieves a minimal single-core computation cost lower than 25 ms even for $N = 1$ billion records. The communication cost of PRC is sublinear in the database size, $O(\sqrt{N})$, as shown in §5.3, and concretely requires about $3\sqrt{N}$ AND gates and two conversion gates for 1-bit and $\lg(N)$ -bit values in our revocation system. We use pairwise broadcast to reconstruct Beaver triples and daBits among n PRC servers, which incurs a quadratic cost in n , but has low impact in practice since n is usually small. As uploads and downloads are symmetric, we report only the one-way communication cost per PRC server.

The client cost is minimal and independent of the number of revocations—revocation simply changes values in the database with no impact on size—requiring $36n + 72$ Bytes of communication and negligible computation. Clients send a query consisting of secret shares of randomness rnd (32 B) and index idx (4 B) to every one of n PRC servers. A single server responds with a token containing the signature (64 B) and auxiliary data (an 8 B timestamp); we omit the commitment since the client knows rnd and the record value, and can locally compute the commitment and verify the signature.

Monetary cost. We use Amazon EC2 pricing [1] to estimate the cost of operating a PRC server. Each server uses a single CPU core and less than 2 GiB of RAM, so we consider low-end EC2 instances priced at \$0.04 per computation hour, with free inbound data transfer and \$0.09 per 1 GiB of outbound traffic. We report the estimated cost under this model in the right plot of Figure 3. The cost scales linearly with the number of records. Our protocol is highly cost-effective, processing *one million requests* for under \$2.5 per PRC server.

Presenting credentials. Adding revocation support introduces the overhead of obtaining a PRC token and linking it to the credential via a proof over the equality of the committed index. As shown in §5.3, the PRC cost is minimal for clients, and the same holds for the proof, which can be realized using a single discrete-log equality proof. The verifier overhead is limited to verifying a signature and the equality proof, both small and comparable to a non-anonymous revocation check. Since these costs are constant and dominated by the underlying credential and signature schemes, we omit benchmarks for clients and verifiers.

Comparison. We compare our work with ALLOSAUR [53] and the PIR-based baseline from §5. ALLOSAUR is a credential revocation scheme with a threshold threat model, but with cost linear in the number of revocations. We provide a detailed comparison in §F for varying number of credentials and revocations. Since ALLOSAUR does not scale for 10 million or more revocations, we extrapolate its server runtime for $N = 1$ billion credentials and a 1% revocation rate to 2663 seconds, while our solution requires only 10.294 ms. We instantiate the PIR baseline with the state-of-the-art single-server (via Hintless [59]) and multi-server (via DPF [10]) PIR schemes with 96 byte tokens $\text{CD}[i] = (\sigma_i, \text{rnd}_i)$. For multi-party settings, we use $n = 2$ servers and report per server revocation cost in Figure 4. As expected, all costs grow linearly with the number of credentials. Existing PIR implementations cannot handle the large databases required in our setting (e.g., 96 GiB for one billion credentials), so we extrapolate costs when systems crash and denote these regions with dashed lines. For $N = 1$ billion credentials, our solution improves revocation computation cost by a factor of $2700\times$ over [10] and $900\times$ over [59]. Excluding the prohibitive cost of maintaining such large and frequently changing databases.

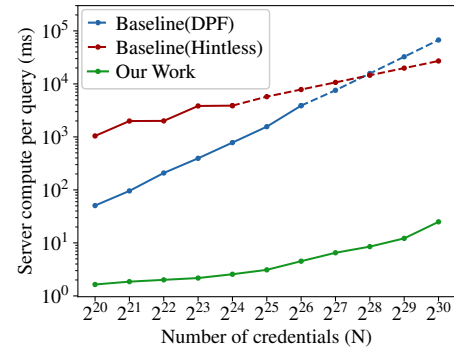


Figure 4: Per-server compute for a query of the PIR baselines, instantiated with Hintless [59] and a DPF [10], varying the number of credentials (N).

8 Conclusion

We have presented an anonymous credential revocation system that satisfies all requirements defined in §4.2. We have shown that our scheme achieves the revocation functionality (RQ.F1) and the privacy properties in §5.5. We now demonstrate that the remaining requirements are also met.

Functionality. In our system, credentials can be dynamically revoked with a single bit-flip, with no overhead for clients or verifiers (RQ.F2). We achieve latency below 400 ms with precise timestamps and full privacy, eliminating long token validity periods and enabling real-time revocation (RQ.F3).

Efficiency. Our system scales to $N = 1$ billion credentials with under 25 ms of computation per RA, far exceeding the requirement to support 100 million users (RQ.D1). As we do not amortize costs, our system also satisfies single-query efficiency (RQ.D3). Requesting a non-revocation token requires negligible computation and 144 B of transfer from clients in a two-server setting, overshadowed by TLS overhead. Token presentation is comparable to non-anonymous revocation, adding a minimal overhead of verifying a signature and a discrete-log equality proof, thus supporting lightweight clients and verifiers (RQ.D2).

Acknowledgment. The authors have used LLM-based tools to refine the text and improve flow, conciseness, and phrasing. This project received funding from the ERC under the EU’s research and innovation programs Horizon Europe (PRIV-TOOLS/101124778 and CRYPTOLAYER/101044770). It was co-funded by the DFG within SFB 1119 CROSS-ING/236615297, and supported by BMFTR and HMWK within ATHENE.

Ethical Considerations

We provide a stakeholder-based ethical analysis below.

Stakeholder Identification and Impacts. Entities that may

be impacted by our research include the research team, the research society as a whole, users of anonymous credential systems, and finally, operators, including governments, of (anonymous) credential systems. For each of the stakeholders, we consider the ethical implications of our work, using the principles listed in the Menlo Report.

Ethical Principles. We consider the principles listed in the Menlo Report to argue about the ethical impact of our work.

Beneficence We propose a new system with beneficial properties, which we assess using benchmarks. All of our benchmarks were conducted in a controlled environment and did not impair any real humans or system users in any way.

Respect for Persons We did not interfere with the personal rights of any of the stakeholders. All experiments were conducted in a controlled environment using synthetic data and do not process, collect, or analyze any personal or otherwise sensitive information.

Justice The research outcomes do not exclude or disadvantage any specific group. Our prototype code is open-source, which enables others to build on top of our results.

Respect for Law and Public Interest Our work does not violate any laws or regulations. Our tests were run using open-source software or using our own implementation. Our research helps to mitigate accountability issues of anonymous credentials, which protect users from surveillance and support users' right to privacy. Our research not only follows European regulation, but it also helps to achieve the protection measures desired in the regulation, while improving the scalability and efficiency.

Harms and Mitigations. This work is purely constructive and does not involve any attacks, exploitation of vulnerabilities, or interaction with deployed systems or real users. For the research team, the research society, and the users of anonymous credential systems, there is no harm in publishing this work. For operators of anonymous credential systems, there may be financial considerations necessary when deciding whether to implement our work in a production environment. Additional infrastructure may be required, which may be supplied by the credential operator. We believe that a more efficient system, potentially leading to a larger adoption, is only in the best interest of the credential operator.

Decision to Proceed with Publication. In summary, this work does not raise any ethical concerns. This work addresses anonymous revocation, which is a critical challenge in anonymous credential systems. We improve on both accountability by enabling authorities to revoke invalid or fraudulent credentials in a right-respecting manner and users to revoke lost or stolen credentials while keeping the strong privacy of anonymous credentials intact. As this work benefits users and society without creating any major risk, we have decided to conduct and publish our findings.

Open Science

Our prototype is available at <https://zenodo.org/records/20433788>. It implements our PRC construction and includes a Docker container for benchmarking. We fully reproduce the PRC cost measurements shown in Figure 3. Presenting a credential with non-revocation requires generating an anonymous credential presentation together with a zero-knowledge proof of equality of discrete logarithm in addition to obtaining a non-revocation token via PRC. This additional constant client cost depends only on the credential scheme and can be added externally. For comparisons with the naive PIR approach (cf. Figure 4) and ALLOSAUR (cf. Figure 5), we document the external runs and cost extrapolations without including third-party repositories in our artifact.

References

- [1] Amazon Web Services. Amazon EC2 Pricing, 2025. <https://aws.amazon.com/ec2/pricing/>, Accessed: 2025-06-04.
- [2] Sebastian Angel, Hao Chen, Kim Laine, and Srinath T. V. Setty. PIR with compressed queries and amortized query processing. In *IEEE Symposium on Security and Privacy*, pages 962–979. IEEE Computer Society, 2018.
- [3] Toshinori Araki, Jun Furukawa, Yehuda Lindell, Ariel Nof, and Kazuma Ohara. High-throughput semi-honest secure three-party computation with an honest majority. In *CCS*, pages 805–817. ACM, 2016.
- [4] Man Ho Au, Willy Susilo, and Yi Mu. Constant-size dynamic k -taa. In *SCN*, volume 4116 of *Lecture Notes in Computer Science*, pages 111–125. Springer, 2006.
- [5] Foteini Baldimtsi, Jan Camenisch, Maria Dubovitskaya, Anna Lysyanskaya, Leonid Reyzin, Kai Samelin, and Sophia Yakoubov. Accumulators with applications to anonymity-preserving revocation. In *EuroS&P*, pages 301–315. IEEE, 2017.
- [6] Carsten Baum, Olivier Blazy, Jan Camenisch, Jaap-Henk Hoepman, Eysa Lee, Anja Lehmann, Anna Lysyanskaya, René Mayrhofer, Hart Montgomery, Ngoc Khanh Nguyen, et al. Cryptographers' feedback on the eu digital identity's arf. *Tech. Rep.*, 2024.
- [7] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *CRYPTO*, volume 576 of *Lecture Notes in Computer Science*, pages 420–432. Springer, 1991.
- [8] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. In *CHES*, volume 6917 of *Lecture Notes in Computer Science*, pages 124–142. Springer, 2011.

- [9] Alexandra Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-diffie-hellman-group signature scheme. In *Public Key Cryptography*, volume 2567 of *Lecture Notes in Computer Science*, pages 31–46. Springer, 2003.
- [10] Dan Boneh, Elette Boyle, Henry Corrigan-Gibbs, Niv Gilboa, and Yuval Ishai. Lightweight techniques for private heavy hitters. In *SP*, pages 762–776. IEEE, 2021.
- [11] Elette Boyle, Niv Gilboa, Yuval Ishai, and Victor I. Kolobov. Information-theoretic distributed point functions. In *ITC*, volume 230 of *LIPICs*, pages 17:1–17:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.
- [12] Elette Boyle, Niv Gilboa, Yuval Ishai, and Victor I. Kolobov. Programmable distributed point functions. In *CRYPTO*, volume 13510 of *Lecture Notes in Computer Science*, pages 121–151. Springer, 2022.
- [13] Zvika Brakerski and Vinod Vaikuntanathan. Fully homomorphic encryption from ring-lwe and security for key dependent messages. In *CRYPTO*, volume 6841 of *Lecture Notes in Computer Science*, pages 505–524. Springer, 2011.
- [14] Stefan Brands, Liesje Demuynck, and Bart De Decker. A practical system for globally revoking the unlinkable pseudonyms of unknown users. In *ACISP*, volume 4586 of *Lecture Notes in Computer Science*, pages 400–415. Springer, 2007.
- [15] Alexander Burton, Samir Jordan Menon, and David J. Wu. Respire: High-rate PIR for databases with small records. In *CCS*, pages 1463–1477. ACM, 2024.
- [16] Jan Camenisch, Manu Drijvers, and Jan Hajny. Scalable revocation scheme for anonymous credentials based on n-times unlinkable proofs. In *WPES@CCS*, pages 123–133. ACM, 2016.
- [17] Jan Camenisch, Manu Drijvers, and Anja Lehmann. Anonymous attestation using the strong diffie hellman assumption revisited. In *TRUST*, volume 9824 of *Lecture Notes in Computer Science*, pages 1–20. Springer, 2016.
- [18] Jan Camenisch, Markulf Kohlweiss, and Claudio Soriente. An accumulator based on bilinear maps and efficient revocation for anonymous credentials. In *Public Key Cryptography*, volume 5443 of *Lecture Notes in Computer Science*, pages 481–500. Springer, 2009.
- [19] Jan Camenisch, Stephan Krenn, Anja Lehmann, Gert Læssøe Mikkelsen, Gregory Neven, and Michael Østergaard Pedersen. Formal treatment of privacy-enhancing credential systems. In *SAC*, volume 9566 of *Lecture Notes in Computer Science*, pages 3–24. Springer, 2015.
- [20] Jan Camenisch and Anna Lysyanskaya. An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In *EUROCRYPT*, volume 2045 of *Lecture Notes in Computer Science*, pages 93–118. Springer, 2001.
- [21] Jan Camenisch and Anna Lysyanskaya. Dynamic accumulators and application to efficient revocation of anonymous credentials. In *CRYPTO*, volume 2442 of *Lecture Notes in Computer Science*, pages 61–76. Springer, 2002.
- [22] Jan Camenisch and Anna Lysyanskaya. A signature scheme with efficient protocols. In *SCN*, *Lecture Notes in Computer Science*. Springer, 2002.
- [23] Jan Camenisch and Anna Lysyanskaya. Signature schemes and anonymous credentials from bilinear maps. In *CRYPTO*, volume 3152 of *Lecture Notes in Computer Science*, pages 56–72. Springer, 2004.
- [24] David Chaum. Blind signatures for untraceable payments. In *CRYPTO*, pages 199–203. Plenum Press, New York, 1982.
- [25] Benny Chor, Oded Goldreich, Eyal Kushilevitz, and Madhu Sudan. Private information retrieval. In *FOCS*, pages 41–50. IEEE Computer Society, 1995.
- [26] Hien Chu, Paul Gerhart, Tim Ruffing, and Dominique Schröder. Practical schnorr threshold signatures without the algebraic group model. In *CRYPTO*, volume 14081 of *Lecture Notes in Computer Science*, pages 743–773. Springer, 2023.
- [27] Henry Corrigan-Gibbs and Dmitry Kogan. Private information retrieval with sublinear online time. In *EUROCRYPT*, volume 12105 of *Lecture Notes in Computer Science*, pages 44–75. Springer, 2020.
- [28] Claude Crépeau. Verifiable disclosure of secrets and applications (abstract). In *EUROCRYPT*, volume 434 of *Lecture Notes in Computer Science*, pages 150–154. Springer, 1989.
- [29] Claude Crépeau, Jeroen van de Graaf, and Alain Tapp. Committed oblivious transfer and private multi-party computation. In *CRYPTO*, volume 963 of *Lecture Notes in Computer Science*, pages 110–123. Springer, 1995.
- [30] Ivan Damgård, Daniel Escudero, Tore Kasper Frederiksen, Marcel Keller, Peter Scholl, and Nikolaj Volgushev. New primitives for actively-secure MPC over rings with applications to private machine learning. In

IEEE Symposium on Security and Privacy, pages 1102–1120. IEEE, 2019.

- [31] Alex Davidson, Gonçalo Pestana, and Sofia Celi. FrodoPIR: Simple, scalable, single-server private information retrieval. *Proc. Priv. Enhancing Technol.*, 2023(1):365–383, 2023.
- [32] Henry de Valence, Jack Grigg, Mike Hamburg, Isis Lovecruft, George Tankersley, and Filippo Valsorda. The ristretto255 and decaf448 groups. RFC 9496, Internet Engineering Task Force, 2023.
- [33] Daniel Demmler, Amir Herzberg, and Thomas Schneider. RAID-PIR: practical multi-server PIR. In *CCSW*, pages 45–56. ACM, 2014.
- [34] Moumita Dutta, Chaya Ganesh, Sikhar Patranabis, and Nitin Singh. Compute, but verify: Efficient multiparty computation over authenticated inputs. In *ASIACRYPT*, volume 15489 of *Lecture Notes in Computer Science*, pages 133–166. Springer, 2024.
- [35] Zeev Dvir and Sivakanth Gopi. 2-server PIR with sub-polynomial communication. *J. ACM*, 63(4):39:1–39:15, 2016.
- [36] Kasra Edalatnejad, Sebastian Faust, Jonas Hofmann, Philipp-Florens Lehwalder, and Thomas Schneider. Do you need a receipt? anonymous credential revocation at continental scale via private record certification. *IACR Cryptol. ePrint Arch.*, 2026.
- [37] Daniel Escudero, Satrajit Ghosh, Marcel Keller, Rahul Rachuri, and Peter Scholl. Improved primitives for MPC over mixed arithmetic-binary circuits. In *CRYPTO*, volume 12171 of *Lecture Notes in Computer Science*, pages 823–852. Springer, 2020.
- [38] European Data Protection Supervisor. TechDispatch #3/2025 - Digital Identity Wallets, 2025. https://www.edps.europa.eu/data-protection/our-work/publications/techdispatch/2025-12-15-techdispatch-32025-digital-identity-wallets_en, Accessed: 2026-03-21.
- [39] European Parliament and Council of the European Union. Regulation (EU) 2024/1183 of 11 april 2024 amending regulation (EU) no 910/2014 as regards establishing the european digital identity framework, April 2024. Official Journal of the European Union, L 202401183. ELI: <http://data.europa.eu/eli/reg/2024/1183/oj>.
- [40] Juan A. Garay. Efficient and universally composable committed oblivious transfer and applications. In *TCC*, volume 2951 of *Lecture Notes in Computer Science*, pages 297–316. Springer, 2004.
- [41] Craig Gentry and Shai Halevi. Compressible FHE with applications to PIR. In *TCC*, volume 11892 of *Lecture Notes in Computer Science*, pages 438–464. Springer, 2019.
- [42] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In *CRYPTO*, volume 8042 of *Lecture Notes in Computer Science*, pages 75–92. Springer, 2013.
- [43] Niv Gilboa and Yuval Ishai. Distributed point functions and their applications. In *EUROCRYPT*, volume 8441 of *Lecture Notes in Computer Science*, pages 640–658. Springer, 2014.
- [44] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In *STOC*, pages 218–229. ACM, 1987.
- [45] Michael T. Goodrich, Roberto Tamassia, and Jasminka Hasic. An efficient dynamic and distributed cryptographic accumulator. In *ISC*, volume 2433 of *Lecture Notes in Computer Science*, pages 372–388. Springer, 2002.
- [46] Daniel Günther, Maurice Heymann, Benny Pinkas, and Thomas Schneider. Gpu-accelerated PIR with client-independent preprocessing for large-scale applications. In *USENIX Security Symposium*, pages 1759–1776. USENIX Association, 2022.
- [47] Daniel Günther, Maurice Heymann, Benny Pinkas, and Thomas Schneider. Gpu-accelerated PIR with client-independent preprocessing for large-scale applications. In *USENIX Security Symposium*, pages 1759–1776. USENIX Association, 2022.
- [48] Iftach Haitner, Jonathan J. Hoch, and Gil Segev. A linear lower bound on the communication complexity of single-server private information retrieval. In *TCC*, volume 4948 of *Lecture Notes in Computer Science*, pages 445–464. Springer, 2008.
- [49] Mike Hamburg. Decaf: Eliminating cofactors through point compression. In *CRYPTO*, volume 9215 of *Lecture Notes in Computer Science*, pages 705–723. Springer, 2015.
- [50] Lukas Helming, Daniel Kales, Sebastian Ramacher, and Roman Walch. Multi-party revocation in sovryn: Performance through distributed trust. In *CT-RSA*, volume 12704 of *Lecture Notes in Computer Science*, pages 527–551. Springer, 2021.

- [51] Alexandra Henzinger, Emma Dauterman, Henry Corrigan-Gibbs, and Nickolai Zeldovich. Private web search with tiptoe. In *SOSP*, pages 396–416. ACM, 2023.
- [52] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. One server for the price of two: Simple and fast single-server private information retrieval. In *USENIX Security Symposium*, pages 3889–3905. USENIX Association, 2023.
- [53] Samuel Jaques, Hart Montgomery, and Michael Lodder. ALLOSAUR: accumulator with low-latency oblivious sublinear anonymous credential updates with revocations. In *AsiaCCS*. ACM, 2024.
- [54] Simon Josefsson and Ilari Liusvaara. Edwards-curve digital signature algorithm (eddsa). RFC 8032, Internet Engineering Task Force, January 2017.
- [55] Chelsea Komlo and Ian Goldberg. FROST: flexible round-optimized schnorr threshold signatures. In *SAC*, volume 12804 of *Lecture Notes in Computer Science*, pages 34–65. Springer, 2020.
- [56] Evan Krul, Hye-young Paik, Sushmita Ruj, and Salil S. Kanhere. Sok: Trusting self-sovereign identity. *Proc. Priv. Enhancing Technol.*, 2024(3):297–313, 2024.
- [57] Maximilian Lam, Jeff Johnson, Wenjie Xiong, Kiwan Maeng, Udit Gupta, Yang Li, Liangzhen Lai, Ilias Leontiadis, Minsoo Rhu, Hsien-Hsin S. Lee, Vijay Janapa Reddi, Gu-Yeon Wei, David Brooks, and G. Edward Suh. Gpu-based private information retrieval for on-device machine learning inference. In *ASPLOS*, pages 197–214. ACM, 2024.
- [58] Ben Laurie. Certificate transparency. *Commun. ACM*, 57(10):40–46, 2014.
- [59] Baiyu Li, Daniele Micciancio, Mariana Raykova, and Mark Schultz. Hintless single-server private information retrieval. In *CRYPTO*, volume 14928 of *Lecture Notes in Computer Science*, pages 183–217. Springer, 2024.
- [60] Dave Longley, Manu Sporny, and Orie Steele. Bitstring status list v1.0. W3c recommendation, World Wide Web Consortium (W3C), 2025.
- [61] Carlo Mazzocca, Abbas Acar, Selcuk Uluagac, Rebecca Montanari, Paolo Bellavista, and Mauro Conti. A survey on decentralized identifiers and verifiable credentials. *IEEE Communications Surveys & Tutorials*, 2025.
- [62] Carlos Aguilar Melchor, Joris Barrier, Laurent Fousse, and Marc-Olivier Killijian. XPIR : Private information retrieval for everyone. *Proc. Priv. Enhancing Technol.*, 2016(2):155–174, 2016.
- [63] Samir Jordan Menon and David J. Wu. SPIRAL: fast, high-rate single-server PIR via FHE composition. In *SP*, pages 930–947. IEEE, 2022.
- [64] Samir Jordan Menon and David J. Wu. SPIRAL: fast, high-rate single-server PIR via FHE composition. In *SP*, pages 930–947. IEEE, 2022.
- [65] Samir Jordan Menon and David J. Wu. YPIR: high-throughput single-server PIR with silent preprocessing. In *USENIX Security Symposium*. USENIX Association, 2024.
- [66] Muhammad Haris Mughees, Hao Chen, and Ling Ren. Onionpir: Response efficient single-server PIR. In *CCS*, pages 2292–2306. ACM, 2021.
- [67] Toru Nakanishi, Hiroki Fujii, Yuta Hira, and Nobuo Funabiki. Revocable group signature schemes with constant costs for signing and verifying. 5443:463–480, 2009.
- [68] Lan Nguyen. Accumulators from bilinear pairings and applications. In *CT-RSA*, volume 3376 of *Lecture Notes in Computer Science*, pages 275–292. Springer, 2005.
- [69] Andrew Park, Trevor Leong, Francisco Maturana, Wenting Zheng, and K. V. Rashmi. Communication-efficient, fault tolerant PIR over erasure coded storage. In *SP*, pages 4331–4347. IEEE, 2024.
- [70] Torben P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *CRYPTO*, volume 576 of *Lecture Notes in Computer Science*, pages 129–140. Springer, 1991.
- [71] David Pointcheval and Olivier Sanders. Short randomizable signatures. In *CT-RSA*, volume 9610 of *Lecture Notes in Computer Science*, pages 111–126. Springer, 2016.
- [72] David Pointcheval and Jacques Stern. Security arguments for digital signatures and blind signatures. *J. Cryptol.*, 13(3):361–396, 2000.
- [73] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *STOC*, pages 84–93. ACM, 2005.
- [74] Leandro Rometsch, Philipp-Florens Lehwalder, Anh-Tu Hoang, Dominik Kaaser, and Stefan Schulte. Uppr: Universal privacy-preserving revocation. In *2025 IEEE International Conference on Blockchain (Blockchain)*, pages 161–170. IEEE, 2025.
- [75] Dragos Rotaru and Tim Wood. Marbled circuits: Mixing arithmetic and boolean circuits with active security. In *INDOCRYPT*, volume 11898 of *Lecture Notes in Computer Science*, pages 227–249. Springer, 2019.

- [76] Claus-Peter Schnorr. Efficient identification and signatures for smart cards. In *CRYPTO*, volume 435 of *Lecture Notes in Computer Science*, pages 239–252. Springer, 1989.
- [77] Reza Soltani, Uyen Trang Nguyen, and Aijun An. A survey of self-sovereign identity ecosystem. *Security and Communication Networks*, 2021, 2021.
- [78] Manu Sporny, Dave Longley, David Chadwick, and Ivan Herman. Verifiable credentials data model v2.0. W3C recommendation, World Wide Web Consortium, 2025.
- [79] Manu Sporny, Dave Longley, Markus Sabadello, Drummond Reed, Orie Steele, and Christopher Allen. Decentralized identifiers (DIDs) v1.0. W3C recommendation, World Wide Web Consortium, 2022.
- [80] Stefano Tessaro and Chenzhi Zhu. Revisiting BBS signatures. In *EUROCRYPT*, volume 14008 of *Lecture Notes in Computer Science*, pages 691–721. Springer, 2023.
- [81] Patrick P. Tsang, Man Ho Au, Apu Kapadia, and Sean W. Smith. Blacklistable anonymous credentials: blocking misbehaving users without ttps. In *CCS*, pages 72–81. ACM, 2007.
- [82] Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *FOCS*, pages 162–167. IEEE Computer Society, 1986.

A Zero-Knowledge Proof Relations

We formally define the proof relations in the full version [36].

B Extensions of our AC Revocation System

We discuss three extensions of our anonymous credential revocation system in §6: enabling offline clients to delegate contacting RAs to the verifier, achieving multi-show presentations of non-revocation tokens, and database privacy.

Extension with offline clients. Real-time revocation (see RQ.F3) requires connection to the RAs. However, as long as the verifier can reach the RAs, our construction can support offline clients that perform a one-round presentation with a non-revocation check and no privacy degradation. This improves support for constrained clients without internet access, such as scenarios with a phone without mobile data.

Observe that in a usual presentation with a non-revocation check, a client 1) performs a PRC request to obtain a non-revocation token and 2) generates a zero-knowledge proof linking the token to its credential. In this proof, the client only needs to know the commitment value co_{rec} inside the token, but does not need to prove over the token (i.e., signature)

itself, which is revealed to the verifier in the clear. Since the commitment randomness is chosen by the client, and it can assume that its credential is valid (i.e., $\mathbf{D}[\text{idx}] = 1$), the client can pre-compute the commitment without actually receiving the non-revocation token. Further, processing the PRC response does not require any private client input and can be fully revealed to the verifier.

In an offline setting, the client can thus prepare the PRC request, encrypt each query under the public key of the corresponding RA, and delegate the interaction with the RAs to the verifier. By also providing the zero-knowledge proof over the pre-computed commitment, the verifier can check the presentation without a further round with the client. To prevent replay attacks and in extension prevent a verifier from repeatedly checking the revocation status of a client, we encrypt add a nonce to the queries. We further assume that the encryption scheme provides confidentiality and integrity, binding the nonce to its query. If one of the RAs receives a nonce twice, they can abort the protocol. Due to the integrity protection, no verifier can change the nonce associated with a query. In the following, we assume that each RA is equipped with a public key encryption key pair $(\text{pk}_i, \text{sk}_i)$. The offline presentation and verification proceed as follows:

- $\text{opt} \leftarrow \text{OfflinePresentNonRev}(\sigma_{\text{cred}}, (a_i)_{i \in [m]}, \text{Rvd}, \text{vk}_{\mathcal{R}})$: The client produces an offline presentation opt for credential σ_{cred} with revocation index idx as follows: 1) compute a PRC request $((\text{qry}_i)_{i \in [n]}, \text{qst}=\text{rnd}) \leftarrow \text{Request}(\text{idx}, \sigma_{\text{auth}})$ and its corresponding commitment $\text{co}_{\text{rec}} \leftarrow \text{Com.Commit}([\text{idx}, \text{rec}=1]; \text{rnd})$; 2) choose a nonce η and encrypt each query under the corresponding PRC server’s public key, $\text{eqry}_i \leftarrow \text{Enc}(\text{pk}_i, (\text{qry}_i, \eta))$; 3) compute the zero-knowledge proof π_{rev} for relation R_{rev} (see §A) using the pre-computed co_{rec} ; and 4) output $\text{opt} := ((\text{eqry}_i)_{i \in [n]}, \text{co}_{\text{rec}}, \pi_{\text{rev}})$.
- $0/1 \leftarrow \text{OfflineVerifyNonRev}(\text{vk}_I, \text{vk}_{\mathcal{R}}, \text{opt}, (a_i)_{i \in \text{Rvd}})$: The verifier combines the PRC interaction with presentation verification as follows: 1) the verifier parses $\text{opt} := ((\text{eqry}_i)_{i \in [n]}, \text{co}_{\text{rec}}, \pi_{\text{rev}})$ and sends each encrypted query eqry_i to the corresponding RA; 2) each RA acting as a PRC server decrypts its query $(\text{qry}_i, \eta) \leftarrow \text{Dec}(\text{sk}_i, \text{eqry}_i)$, computes the response $(\sigma_{\text{rec}}, \text{time}) \leftarrow \text{Response}_S((\text{sk}_i, \text{qry}_i, \mathbf{D})_{i \in [n]}, \perp)$, and send it to the verifier; 3) the verifier forms $\text{pt} := (\sigma_{\text{rec}}, \text{co}_{\text{rec}}, \text{time}, \pi_{\text{rev}})$ and accepts iff $\text{VerifyNonRev}(\text{vk}_I, \text{vk}_{\mathcal{R}}, \text{pt}, (a_i)_{i \in \text{Rvd}})$ accepts.

Correctness is straightforward: if the client’s credential is not revoked, the pre-computed commitment matches the PRC response, and verification accepts. The security argument follows the base construction. Compared with the base version, there are two differences. First, the client cannot verify the PRC response and therefore cannot detect that its credential is revoked and abort contacting the verifier. This is an inherent limit of one-round offline presentations; a two-round variant could trivially solve this. Second, the verifier learns

the encrypted queries eqry_i in addition to pt . The encryption ensures that the verifier query content remains confidential, and nonces prevent reusing queries.

Extension with multi-show presentations. We give a full description of this extension in the full version [36].

Extension with database privacy. As the credentials in our system σ_{cred} already contain the revocation index, one can similarly use it as the authorization credential σ_{auth} to achieve database privacy as described in the extended PRC construction §5.4. Thus, during PresentNonRev , the client uses its credential σ_{cred} as the authorization credential when generating the PRC request. Note that the authorization proof π_{auth} for the relation R_{auth} (see §A) is essentially constructed the same way as the proof of non-revocation for the presentation π_{cred} ; both prove knowledge of a signature containing the same index as in a given commitment. The only difference is that the presentation proof π_{cred} may prove additional statements about other attributes of the credential.

C Security of our PRC Scheme

We give formal definitions and proof of our PRC scheme in the full version [36].

D Security of our AC Revocation Scheme

We give formal definitions and proof of our revocation scheme in the full version [36].

E Pre-processing and correlated randomness generation

Efficient SMC protocols heavily rely on preprocessing to reduce their latency and online phase cost. We use Beaver triples [7] for performing AND operations and daBits [37, 75] for Boolean to arithmetic conversions. The majority of MPC protocols have expensive preprocessing phases requiring GiBs of communication per execution. However, our protocol only requires sub-linear preprocessing material. Specifically, in our largest setting with binary credential statuses ($l = 1$) and a billion credentials ($N = 10^9$), generating $\sim 3\sqrt{N}$ Beaver triples and $\lg(N) + l$ daBits require a transfer of 13.5 KiB for all servers combined when generated via a dealer.

Memory. The pre-processing material required for each query is only 37 KiB small. The client is not involved in the MPC computation and pre-processing material is independent of the querier; thus, servers do not need to fetch client dependent material. Reserving 1 GiB of RAM and pre-loading material in the background supports responding to over 28k queries. We assume that correlation material is available in RAM.

Generation. We use a dealer to generate correlations. The dealer exchanges symmetric keys with the parties in the be-

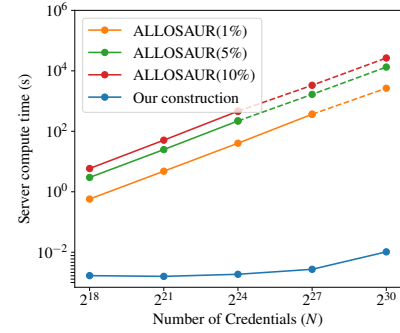


Figure 5: Comparison of server computation of our work and ALLOSAUR for different revocation rates depending on the total number of credentials.

ginning. To generate Beaver triples $(a, b, c = a \wedge b)$, the dealer uses a keyed PRF with the symmetric keys to generate shares of $n - 1$ parties without having to transfer this material. For the last party, the dealer still uses a keyed PRF to randomly select (a_n, b_n) and only computes and sends the final share $c_n = \bigoplus_{i \in [n-1]} c_i \oplus (\bigoplus_{i \in [n]} a_i \wedge \bigoplus_{i \in [n]} b_i)$, requiring 1-bit of transfer per AND triples regardless of the number of parties. We note that keeping triples in RAM requires 3-bits. To generate daBits ($b \in \{0, 1\}, a \in \mathbb{Z}_q = b$), we follow a similar approach of using a keyed PRF to generate $n - 1$ shares and send $(b_n \xleftarrow{\$} \{0, 1\}, a_n = (\bigoplus b_i - \sum a_i) \bmod q)$ to the last party. Sending and storing a daBit requires $(\lg q + 1) = 257$ bits regardless of the number of parties.

Alternative generations. We provide alternative approaches to generate correlated randomness in the full version [36].

F Comparison with ALLOSAUR

ALLOSAUR [53] is the most closely related accumulator-based revocation scheme in an anytrust setting. They utilize MPC to let clients retrieve an updated witness for an accumulator, which accumulates all non-revoked credentials. Unlike our scheme, where updating the revocation database requires only a single bit-flip, each revocation in ALLOSAUR incurs linear work in the total number of credentials for the servers. Client witness update costs additionally scale with the number of revocations since the last update. In Figure 5 we compare the server computation time for varying numbers of credentials and revocation rates, using a RAs consisting of 2 PRC servers. Note that while ALLOSAUR costs depend on the number of revocations, our scheme only depends on the number of credentials. At 1 billion credentials with a 1% revocation rate, ALLOSAUR requires approximately 2663 s per witness update, while our solution requires only 10.294 ms, resulting in a speedup of over 20,000 $\times$. We provide more details on the comparison in the full version [36].